



# Accelerating Algebraic Multigrid Methods via Artificial Neural Networks

Paola F. Antonietti<sup>1</sup> · Matteo Caldana<sup>1</sup> · Luca Dede<sup>1</sup> 

Received: 30 October 2021 / Accepted: 17 September 2022 / Published online: 5 January 2023

© Vietnam Academy of Science and Technology (VAST) and Springer Nature Singapore Pte Ltd. 2023

## Abstract

We present a novel deep learning-based algorithm to accelerate—through the use of Artificial Neural Networks (ANNs)—the convergence of Algebraic Multigrid (AMG) methods for the iterative solution of the linear systems of equations stemming from finite element discretizations of Partial Differential Equations (PDE). We show that ANNs can be successfully used to predict the strong connection parameter that enters in the construction of the sequence of increasingly smaller matrix problems standing at the basis of the AMG algorithm, so as to maximize the corresponding convergence factor of the AMG scheme. To demonstrate the practical capabilities of the proposed algorithm, which we call AMG-ANN, we consider the iterative solution of the algebraic system of equations stemming from finite element discretizations of two-dimensional model problems. First, we consider an elliptic equation with a highly heterogeneous diffusion coefficient and then a stationary Stokes problem. We train (off-line) our ANN with a rich dataset and present an in-depth analysis of the effects of tuning the strong threshold parameter on the convergence factor of the resulting AMG iterative scheme.

**Keywords** Algebraic multigrid (AMG) · Deep learning · Convolutional neural networks · Finite element method · Elliptic PDEs · Stokes problem

**Mathematics Subject Classification (2010)** 65N55 · 65N30 · 65N22 · 68T01

## 1 Introduction

In the last thirty years, there has been an increasing demand for computationally efficient methods to solve sparse linear system of equations stemming from numerical discretization of Partial Differential Equations (PDEs). For real-life problems, the typical size of the resulting algebraic systems makes direct or classical one-level methods impractical and

---

Dedicated to Professor Alfio Quarteroni on the occasion of his 70th birthday.

✉ Luca Dede<sup>\*</sup>  
luca.dede@polimi.it

<sup>1</sup> MOX, Dipartimento di Matematica, Politecnico di Milano, Piazza Leonardo da Vinci 32, 20133 Milano, Italy

hierarchical iterative solvers have been intensively developed and studied. This paper focuses on the Algebraic Multigrid (AMG) method ([61]) for the iterative solution of the symmetric and positive definite systems of equations stemming from Finite Element (FE) approximation ([31, 44, 45]) of elliptic PDEs. One of the main features of AMG is that it is a purely matrix-based approach, thus it does not make use of any geometric information and the hierarchy of operators is constructed directly from the system matrix, provided that the underlying matrix has certain properties, see e.g., [12, 49, 55, 61]. AMG methods can be advantageous whenever geometric multigrid is not a viable option, e.g., whenever the sequence of coarser meshes at the basis of geometric multigrid is not available. AMG and AMG-like approaches have been developed to solve a variety of problems in the context of PDEs-based simulations; here we mention, for example, the AMG method based on element interpolation (AMGe) for solving the discrete equations that arise in Ritz-type finite element methods, [14, 28], Maxwell's equations [34], linear elasticity [8], **Navier–Stokes's equations** [60] and multi-phase porous media [16]. In [7, 9, 17, 40], AMG methods for large-scale supercomputing architectures are presented. In the paper [61] by Xu and Zikatanov, AMG methods are presented and analyzed in a unified framework and an abstract theory for the construction of optimal coarse space as well as quasi-optimal spaces is derived. The abstract framework of [61] covers most of the existing AMG methods, such as classical and energy-minimization AMG, unsmoothed and smoothed aggregation AMG, and spectral AMGe [19]. AMG methods for non-standard FE approximations have been also developed, for example in the context of discontinuous Galerkin methods [4, 10, 52].

The AMG method relies on a set of parameters that defines how to algebraically carry out the coarsening phase. Often their tuning is based on experience and it could be rather inefficient in certain situations. In this paper, we propose **using Machine Learning (ML) and Deep Learning (DL) algorithms to make the choice of the AMG parameters** fully automatic so as to improve the efficiency of the method. The approach that we propose is based on the use of Artificial Neural Networks (ANNs). Artificial Neural Networks are ML and DL models that are nowadays widely used in several problems in image recognition, speech recognition, and natural language processing [22]. The introduction of Convolutional Neural Networks (CNNs) [39] changed modern object recognition process [25, 38, 50]. Today, the most advanced ANNs in image recognition are variations of CNNs: ResNet [26] and SENet [30].

Nowadays, ML and DL models are increasingly being used in scientific computing [43], especially for the numerical approximation of ODEs and PDEs [42]. For example, physics-informed neural networks have been introduced to approximate the solution of PDEs as a meshless method [46, 47] and **ANNs are employed for model order reduction of parameter-dependent PDEs** [21, 29, 48]. ANNs can also be employed to enhance the performance of algorithms and solvers used in “classical” numerical methods for the approximation of PDEs, i.e. as accelerators for scientific computing. In this context, we mention for example: the enhancement of numerical stabilization methods for the FE approximation of advection-dominated differential problems, e.g. in [33, 57]; the use of ANN to optimally select artificial viscosity for discontinuous Galerkin methods in [18]; exploiting CNN for grid refinement in discontinuous Galerkin and virtual element methods in [2, 3]; the hybrid ML-FETI-DP algorithm combining the advantages of adaptive coarse spaces in domain decomposition methods and certain supervised ML techniques that have been proposed in [27]. Moreover, ML techniques and ANNs have already been used to optimize multigrid algorithms, see [23, 24, 35].

In this work, **we make use of ANNs to improve the tuning of the strong threshold parameter that enters in the definition of AMG** so as to improve its performance. In order to test the proposed approach, we consider a two-dimensional elliptic equation with a highly

heterogeneous diffusion coefficient discretized by the FE method. In order to use the sparsity pattern of the underlying matrix as input of the neural network, we introduce a pooling operator. We show how an ANN-enhanced approach can effectively improve the AMG performance. **The performance of the AMG method is measured in two ways:** using the approximate convergence factor and using the elapsed time. We show that these two measures are strictly correlated, this entails that we have a unique way of measuring the performance. We demonstrate that, in some test cases, the value of the strong threshold parameter commonly used in literature can be improved so as to gain efficiency with respect to both measures. In particular, we test different models to tune the hyper-parameters of the model and we report the predictions of the models with the lowest loss. Our model shows very accurate predictions in case of a diffusion coefficient that exhibits “simple” patterns. More complex coefficient distributions seem to work well only if the dataset is reasonably rich, thus more computational work is needed.

The paper is structured as follows. In Section 2 we recall the basic elements of the AMG methods. In Section 3 we introduce the model problem and its FE discretization. In Section 4 we give a brief overview on ANNs. The results of the numerical experiments are showcased in Section 5, namely, we report a wide set of numerical experiments aimed at testing the algorithm’s performance when we vary the strong threshold parameter. Then, in Section 6, we design the architecture of the net, we introduce the pooling operator and test the model. Finally, in Section 7 we draw some conclusions.

## 2 Algebraic Multigrid Methods

In this section, we introduce the main ingredients of AMG methods; we refer the reader to [61] for a comprehensive description. We consider the linear system of equations:

$$\mathbf{A}_h \mathbf{u}_h = \mathbf{f}_h$$

where, for  $n \in \mathbb{N}$ ,  $\mathbf{A}_h \in \mathbb{R}^{n \times n}$  is symmetric and positive definite. Here, and  $\mathbf{u}_h, \mathbf{f}_h \in \mathbb{R}^n$ . Let  $\mathcal{N}_h = \{1, \dots, n\}$  be the set with the indexes of all the variables. The set  $\mathcal{N}_h$  is split into two disjoint subsets  $\mathcal{C}_h$  and  $\mathcal{F}_h$  such that  $\mathcal{N}_h = \mathcal{C}_h \cup \mathcal{F}_h$  and  $\mathcal{C}_h \cap \mathcal{F}_h = \emptyset$ .

Let  $\mathbf{I}_H^h : \mathbb{R}^{n_H} \rightarrow \mathbb{R}^n$  be the interpolation operator that maps coarse level vectors into fine level vectors, and let  $\mathbf{I}_h^H : \mathbb{R}^n \rightarrow \mathbb{R}^{n_H}$  be the restriction operator that maps fine level vectors into coarse level vectors. It is assumed that  $\mathbf{I}_H^h$  can be written as:

$$(\mathbf{I}_H^h \mathbf{e}_H)_i = \begin{cases} (\mathbf{e}_H)_i & \text{if } i \in \mathcal{C}_h, \\ \sum_{k \in \mathcal{P}_i} w_{ij}^h (\mathbf{e}_H)_j & \text{if } i \in \mathcal{F}_h, \end{cases}$$

where  $\mathbf{e}_H \in \mathbb{R}^{n_H}$  is a generic vector,  $\mathcal{P}_i \subset \mathcal{C}_h$ , for all  $i \in \mathcal{F}_h$  is called a set of interpolatory variables for  $i$  and  $w_{ij}^h$  is a set of weights. One way to define  $w_{ij}^h$  is the following. We define the direct neighborhood of a point  $i$  as

$$\text{Neigh}(i) = \{j \neq i : (\mathbf{A}_h)_{ij} \neq 0\}.$$

For  $a \in \mathbb{R}$ , we define its positive and negative part as  $a^+ = \max\{0, a\}$  and  $a^- = \min\{0, -a\}$ , respectively. Similarly, we split  $\mathcal{P}_i$  into two sets

$$\mathcal{P}_i^+ = \mathcal{P}_i \cap \{j \neq i : (\mathbf{A}_h)_{ij} > 0\}, \quad \mathcal{P}_i^- = \mathcal{P}_i \cap \{j \neq i : (\mathbf{A}_h)_{ij} < 0\}.$$

Moreover, [54] shows that the following identity holds

$$(A)_{ii}(\mathbf{e}_h)_i + \alpha_i \sum_{k \in \mathcal{P}_i} (A)_{ik}^-(\mathbf{e}_h)_k + \beta_i \sum_{k \in \mathcal{P}_i} (A)_{ik}^+(\mathbf{e}_h)_k = 0, \quad (1)$$

where the coefficients are given by

$$\alpha_i = \frac{\sum_{j \in \text{Neigh}(i)} (A_h)_{ij}^-}{\sum_{j \in \mathcal{P}_i} (A_h)_{ij}^-}, \quad \beta_i = \frac{\sum_{j \in \text{Neigh}(i)} (A_h)_{ij}^+}{\sum_{j \in \mathcal{P}_i} (A_h)_{ij}^+}.$$

Then, the weights are defined as

$$w_{ik}^h = \begin{cases} -\alpha_i (A_h)_{ik} / (A_h)_{ii}, & k \in \mathcal{P}_i^-, \\ -\beta_i (A_h)_{ik} / (A_h)_{ii}, & k \in \mathcal{P}_i^+, \\ 0 & \text{otherwise.} \end{cases}$$

Since  $A_h$  is symmetric it is also assumed that:

$$I_H^h = (I_H^H)^\top.$$

Then, the coarse-level AMG matrix is defined as  $A_H = I_H^H A_h I_H^h \in \mathbb{R}^{n_H \times n_H}$ . One of the key ingredients of the AMG method consists in the definition of the interpolation operator  $I_H^h$  previously described. The classical coarsening algorithm prescribes to maintain at the coarse level all the strong connections that are defined through a parameter  $\theta$ , called the strong threshold parameter. Its rigorous definition is given in the following [13].

**Definition 1** Let  $A_h \in \mathbb{R}^{n \times n}$ . Given a threshold parameter  $0 < \theta \leq 1$ , the variable  $i$  strongly depends on the variable  $j$  if

$$-(A_h)_{ij} \geq \theta \max_{k \neq i} \{-(A_h)_{ik}\}, \quad i, j = 1, \dots, n.$$

As a matter of fact, performing the  $\mathcal{C}_h/\mathcal{F}_h$  splitting and defining the operators  $I_h^H$  and  $I_H^h$  requires choosing such strong threshold parameter  $\theta$ . Even if the weights  $w_{ij}^h$  do not directly depend on  $\theta$ , the performance of the AMG method will depend on the choice of the threshold parameter, which is empirically made a priori. To show how the choice of  $\theta$  enters in the construction of the operators  $I_h^H$  and  $I_H^h$ , we briefly recall the coarsening algorithm of [54]. We introduce two sets that exploit Definition 1. The first set contains all the indexes  $j$  that are strongly connected to the index  $i$ , i.e.

$$\mathcal{S}_i = \{j \in \text{Neigh}(i) : i \text{ is strongly dependent on } j\}, \quad i = 1, \dots, n.$$

Next, given  $\mathcal{S}_i$ , we introduce  $\mathcal{S}_i^\top$  as:

$$\mathcal{S}_i^\top = \{j \in \mathcal{N}_h : i \in \mathcal{S}_j\}.$$

The coarsening procedure follows this algorithm:

1. Initialize the set of undecided variables  $\mathcal{U}_h \leftarrow \mathcal{N}_h$ .
2. Choose a variable  $i \in \mathcal{U}_h$  such that  $\eta(i) \geq \eta(k) \forall k \in \mathcal{U}_h$ , where

$$\eta(k) = |\mathcal{S}_k^\top \cap \mathcal{U}_h| + 2|\mathcal{S}_k^\top \cap \mathcal{F}_h| \quad \forall k \in \mathcal{U}_h,$$

and  $|\cdot|$  denotes the cardinality.

3. Move the index  $i$  from the set  $\mathcal{U}_h$  to the set  $\mathcal{C}_h$ .
4. Add all the variables  $j \in \mathcal{S}_i^\top$  to the set  $\mathcal{F}_h$ , that is add to the set  $\mathcal{F}_h$  all the variables  $j$  that strongly depend on  $i$ .

Steps (2–4) are repeated until all the variables are either in  $\mathcal{C}_h$  or  $\mathcal{F}_h$ . The measure  $\eta$  is needed to avoid a non-uniform distribution of the variables. In this way, at each iteration, the algorithm selects as  $i$  ( $\mathcal{C}_h$ -variable) the index such that the majority of  $\mathcal{F}_h$ -variables strongly dependent on.

The last ingredient needed to define the AMG methods is a smoothing operator. In general, one iteration of the smoothing can be written as:

$$\mathbf{u}_h^{(k+1)} = S_h \mathbf{u}_h^{(k)} + \mathbf{g}_h, \quad k \geq 0,$$

where  $S_h \in \mathbb{R}^{n \times n}$  denotes the smoothing operator to be properly chosen. Equivalently, it can be written in preconditioned form as:

$$\mathbf{u}_h^{(k+1)} = \mathbf{u}_h^{(k)} + B_h(\mathbf{f}_h - A_h \mathbf{u}_h^{(k)}), \quad k \geq 0, \quad (2)$$

where  $S_h = I_h - B_h A_h$ ,  $\mathbf{g}_h = B_h \mathbf{f}_h$ ,  $B_h = (I_h - S_h) A_h^{-1}$  and  $I_h$  is the identity operator. In practice, either  $(S_h, \mathbf{g}_h)$  or  $B_h$  are given and uniquely identify the smoother. In the following, the notation:

$$\mathbf{u}_h^{(l)} = \text{smooth}^l(A_h, \mathbf{u}_h^{(0)}, \mathbf{f}_h),$$

means that  $\mathbf{u}_h^{(l)}$  is the result of  $l$  steps of (2), starting from an initial vector  $\mathbf{u}_h^{(0)}$ . In Algorithm 1 we report one iteration of the two-level algorithm, where  $\nu_1$  and  $\nu_2$  are the number of smoothing steps that we apply before and after the error correction, respectively. The complete two-level AMG algorithm is outlined in Algorithm 2. As usual, in Algorithm 2,  $tol$  is a user-defined tolerance that is employed as a stopping criterion. Analogously,  $N_{\max}$  is the maximum number of iterations allowed. We notice that Algorithm 1 involves selecting the parameter  $\theta$  a priori.

---

```

1  $\mathbf{u}_h^{(*)} \leftarrow \text{smooth}^{\nu_1}(A_h, \mathbf{u}_h^{(k)}, \mathbf{f}_h)$ 
2  $\mathbf{r}_h \leftarrow \mathbf{f}_h - A_h \mathbf{u}_h^{(*)};$ 
3  $\mathbf{r}_H \leftarrow I_h^H \mathbf{r}_h;$ 
4  $\mathbf{e}_H \leftarrow \text{solve}(A_H, \mathbf{r}_H);$ 
5  $\mathbf{u}_h^{(*)} \leftarrow \mathbf{u}_h^{(*)} + I_H^h \mathbf{e}_H;$ 
6  $\mathbf{u}_h^{(k+1)} \leftarrow \text{smooth}^{\nu_2}(A_h, \mathbf{u}_h^{(*)}, \mathbf{f}_h)$ 

```

---

**Algorithm 1** One Iteration of the two-level AMG method  $\mathbf{u}_h^{(k+1)} = \text{two\_level\_iteration}(\mathbf{u}_h^{(k)}, A_h, \mathbf{f}_h, \nu_1, \nu_2, I_h^H, I_H^h)$ .

As a matter of fact, the two-level AMG Algorithm 2 can be immediately extended to many levels by simply calling recursively Algorithm 2 until a sufficiently coarse level is reached (where a direct solver is employed). For the sake of the analysis carried out in the present paper, we will focus on the two-level method.

---

```

1 perform the  $\mathcal{C}_h/\mathcal{F}_h$ -splitting using  $\theta$ ;
2 build the operators  $\mathbf{I}_h^H, \mathbf{I}_H^h$  using the  $\mathcal{C}_h/\mathcal{F}_h$ -splitting ;
3 while  $k < N_{\max}$  and  $\|\mathbf{A}_h \mathbf{u}_h^{(k)} - \mathbf{f}_h\|/\|\mathbf{f}_h\| < tol$  do
4   |  $\mathbf{u}_h^{(k+1)} \leftarrow \text{two\_level\_iteration}(\mathbf{u}_h^{(k)}, \mathbf{A}_h, \mathbf{f}_h, v_1, v_2, \mathbf{I}_h^H, \mathbf{I}_H^h)$ 
5 end

```

---

**Algorithm 2** Two-Level AMG algorithm  $\mathbf{u}_h^{(k+1)} = \text{AMG}(\mathbf{u}_h^{(0)}, \mathbf{A}_h, \mathbf{f}_h, \theta, v_1, v_2, N_{\max}, tol)$ .

### 3 Model Problems

Throughout this work, we use standard notation for Sobolev spaces [41]. Let  $\Omega$  be an open, bounded domain in  $\mathbb{R}^2$  and let  $\partial\Omega = \overline{\Gamma}_D$ . The first model problem we consider reads:

$$\begin{cases} -\operatorname{div}(\mu(x, y)\nabla u) = f & \text{in } \Omega, \\ u = g_D & \text{on } \Gamma_D, \end{cases} \quad (3)$$

where  $f \in L^2(\Omega)$  is a given forcing term, and  $g_D \in H^{1/2}(\Gamma_D)$  is the given Dirichlet boundary data. The function  $\mu \in L^\infty(\Omega)$  is a positive diffusion coefficient. In this work it will be a piece-wise non-negative constant function. To handle non homogeneous Dirichlet boundary condition we define  $\tilde{u}$  by the means of the lifting  $\tilde{u} = u - \tilde{g}$ , where  $\tilde{g}$  is an extension of  $g_D$  in  $H^1(\Omega)$ . The weak formulation of problem (3) reads:

$$\text{find } \tilde{u} \in H_{\Gamma_D}^1(\Omega) : (\tilde{u}, v) = F(v) \quad \forall v \in H_{\Gamma_D}^1(\Omega), \quad (4)$$

where  $H_{\Gamma_D}^1(\Omega) := \{v \in H^1(\Omega) : v|_{\Gamma_D} = 0\}$  and

$$a(\tilde{u}, v) = \int_{\Omega} \mu \nabla \tilde{u} \cdot \nabla v \, d\Omega, \quad F(v) = \int_{\Omega} f v \, d\Omega - \int_{\Omega} \mu \nabla \tilde{g} \cdot \nabla v \, d\Omega.$$

The well-posedness of problem (4) is given by the Lax–Milgram’s theorem [15].

Now we pass to the FE formulation. We consider a quasi uniform mesh  $\mathcal{T}_h$  of  $\Omega$ . We denote with the parameter  $h > 0$  the mesh size of  $\mathcal{T}_h$  given by  $h = \max_{T \in \mathcal{T}_h} h_T$ , where  $h_T$  is the diameter of the element  $T \in \mathcal{T}_h$ . In our case, we use:

$$V_h = \{v_h \in X_h^1 : v_h = 0 \text{ on } \Gamma_D\},$$

where  $X_h^1 = \{v_h \in C^0(\bar{\Omega}) : v_h|_T \circ F_T \in \mathbb{Q}_1(\hat{\Omega}) \, \forall T \in \mathcal{T}_h\}$ ,  $F_T : \hat{\Omega} \rightarrow T$  is an invertible function that maps the reference square  $\hat{\Omega} = (-1, 1)^2$  to the mesh element  $T$ , and  $\mathbb{Q}_N$  is the space of polynomials with real coefficients and degree less than or equal to  $N$  in each coordinate direction. The finite dimensional formulation of (4) reads:

$$\text{find } \tilde{u}_h \in V_h \text{ s.t.: } a(\tilde{u}_h, v_h) = F(v_h) \quad \forall v_h \in V_h. \quad (5)$$

By setting  $n = \dim(V_h)$ , we denote with  $\{\phi_1, \dots, \phi_n\}$  the FE basis for  $V_h$ . Then, from (5), we obtain the linear system of equations  $\mathbf{A}_h \mathbf{u}_h = \mathbf{f}$ , where:

$$a(\phi_j, \phi_i) = (\mathbf{A}_h)_{ij}, \quad F(\phi_i) = (\mathbf{f})_i, \quad (\mathbf{u}_h)_i = \tilde{u}_i. \quad (6)$$

As a second model problem we consider the Stokes equations. Namely, we are looking for a velocity  $\mathbf{u} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$  and pressure  $p : \mathbb{R}^2 \rightarrow \mathbb{R}$  that satisfy the Stokes equation, which reads

$$\begin{cases} -\nu \Delta \mathbf{u} + \nabla p = \mathbf{0} & \text{in } \Omega, \\ -\operatorname{div} \mathbf{u} = 0 & \text{in } \Omega, \\ \mathbf{u} = \mathbf{0} & \text{on } \Gamma_0, \\ \mathbf{u} = \mathbf{u}_0 & \text{on } \Gamma_{in}, \\ \nu \partial_{\mathbf{n}} \mathbf{u} - p \mathbf{n} = 0 & \text{on } \Gamma_{out}, \end{cases} \quad (7)$$

where  $\mathbf{n}$  denotes the outer normal vector and  $\mathbf{u}_0$  is the parabolic inflow velocity, with maximum  $U > 0$ . Here, we decompose the boundary  $\Gamma$  as  $\Gamma = \Gamma_{in} \cup \Gamma_{out} \cup \Gamma_0$ , where  $\Gamma_{in}$ ,  $\Gamma_{out}$ ,  $\Gamma_0$  are disjoint open sets with positive measure. To guarantee the well-posedness of the problem, we prescribe that  $p \in L_0^2(\Omega)$  i.e. is a  $L^2(\Omega)$  functions with zero average. We introduce the functional spaces:

$$V = \{v \in [H^1(\Omega)]^2 : v|_{\partial\Omega} = 0\}, \quad Q = L_0^2(\Omega),$$

and endow them with the norms  $\|v\|_V = \|v^{1/2} \nabla v\|_{L^2(\Omega)}$ ,  $\|q\|_Q = \|q\|_{L^2(\Omega)}$ . The weak formulation of problem (7) reads: find  $(u, p) \in V \times Q$ , such that

$$a(u, v) + b(p, v) - b(q, u) = 0 \quad \forall (v, q) \in V \times Q$$

where

$$\begin{aligned} a : V \times V &\rightarrow \mathbb{R}, & a(u, v) &= \int_{\Omega} \nu \nabla u : \nabla v, \\ b : Q \times V &\rightarrow \mathbb{R}, & b(p, v) &= - \int_{\Omega} p \operatorname{div} v. \end{aligned}$$

It is well-known that the bilinear form  $b(\cdot, \cdot)$  satisfies a continuous inf-sup condition; see, e.g., [11]. We introduce a uniform quadrilateral mesh  $\mathcal{T}_h$  of  $\Omega$ . Discretizing using the standard polynomial spaces  $V_h = [X_h^2]$ ,  $Q_h = X_h^1$  on  $\mathcal{T}_h$  we obtain the following algebraic formulation

$$\begin{pmatrix} A_h & B_h^\top \\ B_h & 0 \end{pmatrix} \begin{pmatrix} \mathbf{u}_h \\ \mathbf{p}_h \end{pmatrix} = \begin{pmatrix} \mathbf{0} \\ \mathbf{0} \end{pmatrix}, \quad (8)$$

where, setting  $N_h = \dim(V_h)$  and  $M_h = \dim(Q_h)$ ,  $A_h \in \mathbb{R}^{N_h \times N_h}$  and  $B_h \in \mathbb{R}^{M_h \times N_h}$  are the matrix representation of the bilinear forms  $a(\cdot, \cdot)$  and  $b(\cdot, \cdot)$ , respectively.

## 4 Artificial Neural Networks

An artificial neural network, is a regression (or classification) model which given by a function  $\mathcal{F} : \mathbb{R}^N \rightarrow \mathbb{R}^M$  defined as

$$\mathcal{F}(\mathbf{x}; \boldsymbol{\gamma}) = \mathbf{y},$$

where  $\mathbf{x}$  is the input,  $\mathbf{y}$  is the predicted value of the regression and  $\boldsymbol{\gamma}$  is the vector containing all the parameters of the model. The function  $\mathcal{F}$  is the composition of  $K$  functions  $\mathcal{F}^{(k)}$

called *layers*, the number of layers  $K$  is called *depth* of the model. In the case of feed-forward neural network the layer is defined as

$$\begin{cases} \mathbf{a}^{(k)} = \mathbf{W}^{(k)} \mathbf{z}^{(k-1)} + \mathbf{b}^{(k)}, \\ \mathbf{z}^{(k)} = \mathcal{H}^{(k)}(\mathbf{a}^{(k)}) \end{cases}, \quad \text{for } k = 1, \dots, K$$

$$\mathbf{x} = \mathbf{z}^{(0)}, \mathbf{y} = \mathbf{z}^{(K)}, N_0 = N, N_K = M,$$

$$\boldsymbol{\gamma} = \{(\mathbf{W}^{(1)}, \mathbf{b}^{(1)}), \dots, (\mathbf{W}^{(K)}, \mathbf{b}^{(K)})\},$$
(9)

where  $\mathbf{W}^{(k)} \in \mathbb{R}^{N_k \times N_{k-1}}$  (*weights*) and  $\mathbf{b}^{(k)} \in \mathbb{R}^{N_k}$  (*biases*) are the parameters  $\boldsymbol{\gamma}$ , and  $\mathcal{H}^{(k)}(\cdot)$  is a scalar non-linear almost everywhere differentiable function that is applied component-wise to  $\mathbf{a}^{(k)}$  and called *activation function*. The Rectified Linear Unit  $\text{ReLU}(x) = \max\{0, x\}$  is our choice of activation function  $\mathcal{H}^{(k)}(\cdot)$  since, in recent years, it has become very popular due to the fact that it greatly improves the convergence of the stochastic gradient descent algorithm compared to the sigmoid/tanh functions [38]. Moreover, it features lighter computations with a random initialization network as only about half of hidden units have a non-zero output and faster evaluation with respect to the sigmoid/tanh functions. Indeed, in our experiments, employing the ReLU activation function seems to lead to better results with respect to the tanh activation function.

Next, we define the loss function  $\mathcal{L}$ . We assume that a dataset composed by  $P$  couples  $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})$  is available; these are realizations of random variables  $\mathcal{X}, \mathcal{Y}$ . Once defined the ANN architecture, its training boils down to minimize the average training error, namely

$$J(\boldsymbol{\gamma}) = \frac{1}{P} \sum_{i=1}^P \mathcal{L}(\mathcal{F}(\mathbf{x}^{(i)}; \boldsymbol{\gamma}), \mathbf{y}^{(i)}).$$

A typical choice of the loss function  $\mathcal{L}$  that we also use in this paper are the Mean Square Error (MSE) and Mean Absolute Error (MAE).

For determining the parameters  $\boldsymbol{\gamma}$ , we use the Adaptive Moment Estimation (Adam) method [36]. It is a variant of the stochastic gradient descent method that combines the Root Mean Squared propagation (RMSProp) algorithm [58] and momentum method [56] few other significant modifications, namely the momentum is recorded in the history of the gradient and there is a correction term of the bias for the estimation of the first and second order moments of the gradient.

Finally, to prevent overfitting and minimize the generalization error we employ four regularization techniques. Namely, we will always employ an early stopping criterion, namely we stop the training at the point of smallest error with respect to the validation dataset and random parameter initialization [25]. Moreover, we will test dropout, which consists in randomly omitting the weights and biases of some neurons  $(\mathbf{z}^{(k)})_l$  during the training process [53], and batch normalization, a transformation applied at the end of a layer that normalizes its output by the empirical mean and variance of the minibatch [32]. As we will show in the forthcoming sections, the latter seems not to lead to substantial improvements in our model.

As we want to use the matrix of the linear system  $\mathbf{A}_h$  as input of the network we employ CNN. Their characteristic is that the layer takes the form of a cross-convolution between the input and a matrix  $\mathbf{K} \in \mathbb{R}^{D \times D}$ , called *kernel*. Three other hyper-parameters control how the convolution is performed: number of filters, stride and zero-padding size. Moreover, in the last stage of the layer a pooling function is applied. The pooling function is a form of down-sampling that replaces the output of the net at a certain location with a summary statistic of the nearby outputs. The aim of the pooling operation is to control the number of parameters and limit the overfitting. We refer the reader to [22] for more details.

## 5 Numerical Assessment of the Dependence of the Performance of the AMG Method on the Strong Threshold Parameter

In this section we assess the relation between the choice of the strong threshold parameter  $\theta$  and the corresponding performance of the AMG method.

For our model problem (3) we select the diffusion coefficient  $\mu$  to be a piece-wise positive constant function. We assume that  $\mu$  features different patterns, where the domain  $\Omega$  splits into strides or has a checkerboard pattern; see Fig. 1. The value of  $\mu(x, y)$  depends on which “tile”  $(x, y)$  it belongs to, namely

$$\mu(x, y) = \begin{cases} 1 & \text{if } (x, y) \in \Omega_{gray}, \\ 10^\varepsilon & \text{if } (x, y) \in \Omega_{white}, \end{cases} \quad (10)$$

where  $\varepsilon$  is a parameter and  $\Omega_{gray}$  and  $\Omega_{white}$  are shown in Fig. 1. The experiments were carried out so that the exact solution  $u$  of problem (3) is  $u(x, y) = \cos(\pi x) \cos(\pi y)$  for patterns (a) and (b), while  $u(x, y) = \cos(2\pi x) \cos(2\pi y)$  for patterns (c) and (d). Dirichlet boundary conditions are set on the whole boundary  $\partial\Omega$ . Moreover, we employ regular cartesian meshes, so that the discontinuity of  $\mu$  is aligned with mesh elements.

The implementation of the AMG method on which we rely on is the BoomerAMG of the library HYPRE [20]. In particular, we use the AMG method as a preconditioner to accelerate the Conjugate Gradient (CG) iterative method [6]. The simulations were run using deal.II [5] with PETSc [1] on Ubuntu 18.04 LTS with CPU Intel i7-8550U. For sake of simplicity, the computations were carried out in serial. However, even if the choice of  $\theta$  might influence the parallelization, the same approach could be extended also to the parallel case.

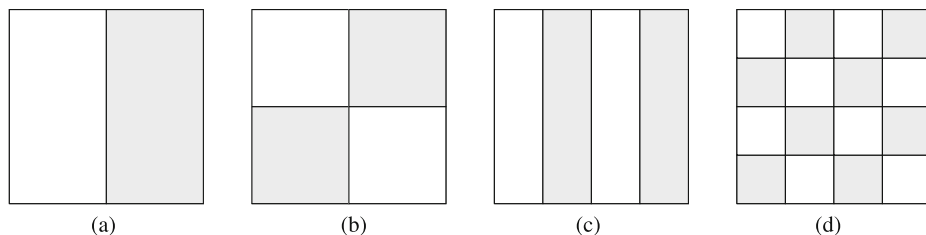
To measure the performance of AMG we employ two performance indexes  $p$ : the elapsed CPU time and the approximate convergence factor  $\rho$ , defined as follows. Let  $\rho^{(k)}$  be defined as

$$\rho^{(k)} = \left( \frac{\|\mathbf{r}^{(k)}\|}{\|\mathbf{r}^{(0)}\|} \right)^{\frac{1}{k}},$$

where  $\mathbf{r}^{(k)}$  is the residual at the  $k$ -th iteration and  $\|\cdot\|$  is the standard euclidean norm. Then, we define  $\rho$  as

$$\rho = \rho^{(N_{it})},$$

where  $N_{it}$  is the number of iterations reached to reduce the (relative) residual below the given tolerance of the linear solver (here it is equal to  $N_{it} = \min_k \{k \in \mathbb{N} \text{ such that } \|\mathbf{r}^{(k)}\| < 10^{-8}\}$ ).



**Fig. 1** Four possible patterns of the diffusion coefficient  $\mu$  of problem (3) on  $\Omega = \Omega_{gray} \cup \Omega_{white} = (-1, 1)^2$ : it is defined such that  $\mu = 1$  on the white tiles  $\Omega_{white}$  and  $\mu = 10^\varepsilon$ ,  $\varepsilon > 0$ , on the gray ones  $\Omega_{gray}$

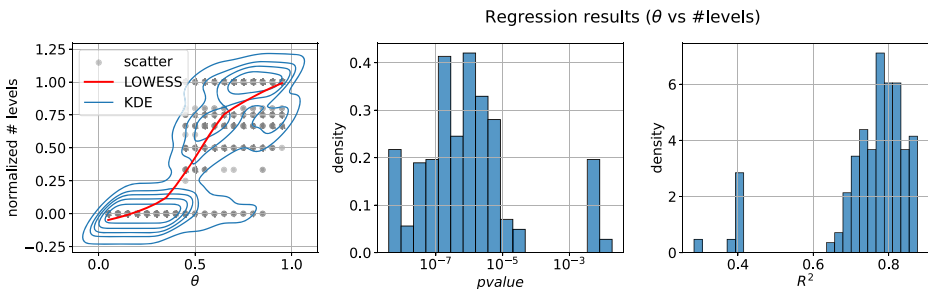
**Table 1** Definition of the quantities employed in the analysis of Section 5.1 and Section 5.3

| Quantity               | Definition                     | Formula                                                                                                                                                          |
|------------------------|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $nn$                   | Number of data points          |                                                                                                                                                                  |
| SSE                    | Sum of squares of errors       | $\sum_{i=1}^{nn} (y^{(i)} - \hat{x}_1 x^{(i)} - \hat{x}_0)^2$                                                                                                    |
| TSS                    | Total sum of squares           | $\sum_{i=1}^{nn} (y^{(i)} - \bar{y})^2$ , $\bar{y} = \frac{1}{nn} \sum_{i=1}^{nn} y^{(i)}$                                                                       |
| SSR                    | Sum of squared residuals       | $\sum_{i=1}^{nn} (\bar{y} - \hat{x}_1 x^{(i)} - \hat{x}_0)^2$                                                                                                    |
| $R^2$                  | Coefficient of determination   | $1 - \text{RSS} / \text{TSS}$                                                                                                                                    |
| F-statistic            | F-statistic of the regression  | $(nn - 2) \text{SSR} / \text{SSE}$                                                                                                                               |
| AIC                    | Akaike's information criterion | $4 - 2 \log(\hat{L})$ , where $\hat{L}$ is the log-likelihood of the model                                                                                       |
| $\text{SE}(\hat{x}_1)$ | Standard error of $\hat{x}_1$  | $\sqrt{\text{SSE} / \sum_{i=0}^{nn} (x^{(i)} - \bar{x})^2 / (nn - 2)}$                                                                                           |
| t-value $\hat{x}_1$    | t-value of $\hat{x}_1$         | $\hat{x}_1 / \text{SE}(\hat{x}_1)$                                                                                                                               |
| p-value $\hat{x}_1$    | p-value of $\hat{x}_1$         | $2 \text{cdf}_{t,1}(- tv_{\hat{x}_1} )$ , where $\text{cdf}_{t,1}$ is the cumulative density function of the Student's t distribution with one degree of freedom |

The analysis aims at assessing the relation between the predictor scalar variable  $x$  and the predicted scalar variable  $y$  given  $nn$  data points  $(x^{(i)}, y^{(i)})$ . We consider the linear model  $y = \hat{x}_1 x + \hat{x}_0$ . We refer to [51] for more details

## 5.1 Relation Between $\theta$ and the Number of Levels

In this section, we show how different choices of  $\theta$  influence the number of levels built by the BoomerAMG algorithm and the corresponding size of the coarsest matrix  $A_h$ . Let us call *test case* a fixed choice of the pattern of the diffusion coefficient  $\mu$ , the coefficient  $\varepsilon$ , and the size of the mesh  $h$ . For each test case, we vary  $\theta$  and record the number of levels used by the AMG algorithm to solve the problem. The scatter in Fig. 2 (left) shows that if the strong threshold is small, namely  $0 < \theta < 0.3$ , the number of levels is constant and it is equal to the minimum number of levels used to solve that test case (see Table 1 for definitions). In Fig. 2 (left) we superimposed a Kernel Density Estimate (KDE), which is an estimate of the density of the distribution from where the samples are drawn that employs a standard normal kernel, and a Locally Weighted Scatterplot Smoothing (LOWESS), which is a locally weighted linear least squares giving more weight to points near the point whose



**Fig. 2** *Left.* A scatter plot of the strong threshold parameter  $\theta$  versus the min-max normalized number of levels of the AMG method. We superimposed a KDE and a LOWESS. *Center and Right.* For each test case in the dataset we perform a least square analysis between  $\theta$  and the corresponding number of levels of the AMG method. *Center.* Histogram of the p-value of the least square analysis. *Right.* Histogram of the coefficient of determination ( $R^2$ ). We refer to Table 1 for the definition of p-value and  $R^2$

response is predicted. We refer to [59] for their precise definition. The LOWESS shows that if  $\theta > 0.3$ , the number of levels increases, almost linearly, as the strong threshold increases. Indeed, the KDE displays higher density spots in the upper right zone. Indeed, a larger value of  $\theta$  means that more connections are kept and the number of unknowns between two connecting levels is only partly reduced. In particular, in 95% of the test cases, the number of levels is a non-decreasing function with respect to  $\theta$ . For each test case, we perform the least square analysis between the value of  $\theta$  and the corresponding number of levels. Figure 2 (center and right) shows that in most of the test cases there is a significant correlation (p-value  $< 10^{-5}$ ) between these two variables.

We carried out the same set of experiments varying the value of  $\theta$  and recording the corresponding size of the coarse matrix  $A_H$  built by BoomerAMG. The results of the least square analysis seems to indicate there is no correlation between these two variables.

## 5.2 Relation Between $\theta$ and $\rho$

In this section we investigate the relation between the strong threshold parameter  $\theta$  and the corresponding approximated convergence factor  $\rho$ . The results reported in Table 2 have been obtained with a diffusion coefficient that has a “strides” pattern (Fig. 1(c)), while Table 3 displays analogous results on the checkerboard pattern (Fig. 1(d)).

We have computed the value of  $\rho$  and the corresponding iteration counts as a function of the value of  $\varepsilon$  of the diffusion coefficient (10) and the mesh size  $h$ . The value of  $\theta$  is kept fixed for each test.

By comparing one test with the others, we can determine if the different value of the strong threshold parameter  $\theta$  has affected the convergence factor  $\rho$  of the linear solver. Twenty-five values of  $\theta$  in  $[0.02, 0.9]$  have been chosen. In Tables 2 and 3 we report the results for three values of  $\theta$  that are representative of the obtained results when  $\theta$  is “small”, “medium” and “large”, namely  $\theta = 0.24, 0.48, 0.72$ . The values of  $\varepsilon$  go from 0.0 (yielding the standard Laplacian problem with uniform diffusion), to 9.5, which produces a quite large discontinuity in the diffusion coefficient  $\mu$ .

From the results of Tables 2 and 3, it is clear that, if the choice of strong threshold  $\theta$  is appropriate, there is almost always uniform convergence, independently of the mesh size  $h$ . This confirms that the AMG method works as expected also with a diffusion coefficient  $\mu$  that presents large discontinuities, provided that  $\theta$  is appropriately chosen.

The results reported in Tables 2 and 3 also show that for large values of the strong threshold parameter ( $\theta = 0.72$ ), the approximate convergence factor  $\rho$  increases, i.e. the convergence properties of the AMG method seem to deteriorate. A possible explanation is the following: as we mentioned in Section 5.1, from Fig. 2 we can conclude that a larger value of  $\theta$  implies that a larger number of levels will be needed by the AMG algorithm. This might lead to a deterioration of the convergence rates.

For the test cases that present less pronounced discontinuities, the value of  $\theta = 0.25$  (which is almost the standard literature value) provides uniform convergence. On the other hand, we notice that in the strongly heterogeneous cases (i.e. when  $\varepsilon$  is large) deviating from the literature value of  $\theta$  can result in a significant improvement in the approximate convergence factor.

The results shown in Fig. 2 seem to indicate that choosing  $\theta$  differently from the standard value suggested in literature does not result in any significant improvement. On the other hand, the plots of the four finest mesh refinements of Fig. 3 reveal that a significant boost in the performance could be obtained. An optimal choice of the strong threshold could bring up to 33% speed-up w.r.t. the default choice of  $\theta = 0.25$ .

**Table 2** Computed values of the approximate convergence factor  $\rho$  and corresponding preconditioned CG iteration counts (between parenthesis) w.r.t. parameters  $\varepsilon$  on rows and mesh size  $h$  on columns

| $\varepsilon \backslash h$                                                                             |     | 1.25e-01 | 6.25e-02  | 3.12e-02  | 1.56e-02  | 7.81e-03  | 3.91e-03  | 1.95e-03  | 9.77e-04  |
|--------------------------------------------------------------------------------------------------------|-----|----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| $\theta = 0.24$<br>   | 0.0 | 0.094(9) | 0.071(8)  | 0.060(8)  | 0.054(8)  | 0.061(9)  | 0.063(9)  | 0.064(9)  | 0.066(10) |
|                                                                                                        | 0.4 | 0.091(9) | 0.069(8)  | 0.059(8)  | 0.057(8)  | 0.061(9)  | 0.062(9)  | 0.063(9)  | 0.066(10) |
|                                                                                                        | 0.8 | 0.087(9) | 0.066(8)  | 0.059(8)  | 0.058(8)  | 0.059(9)  | 0.060(9)  | 0.061(9)  | 0.068(10) |
|                                                                                                        | 1.2 | 0.085(9) | 0.066(8)  | 0.060(8)  | 0.059(8)  | 0.061(9)  | 0.060(9)  | 0.061(9)  | 0.069(10) |
|                                                                                                        | 1.6 | 0.085(9) | 0.065(8)  | 0.061(8)  | 0.067(9)  | 0.062(9)  | 0.061(9)  | 0.062(9)  | 0.070(10) |
|                                                                                                        | 2.0 | 0.084(9) | 0.065(8)  | 0.062(8)  | 0.068(9)  | 0.062(9)  | 0.061(9)  | 0.062(9)  | 0.070(10) |
|                                                                                                        | 2.4 | 0.084(9) | 0.065(8)  | 0.062(8)  | 0.068(9)  | 0.062(9)  | 0.062(9)  | 0.062(9)  | 0.070(10) |
|                                                                                                        | 2.8 | 0.084(9) | 0.065(8)  | 0.062(8)  | 0.069(9)  | 0.062(9)  | 0.062(9)  | 0.063(9)  | 0.070(10) |
|                                                                                                        | 3.5 | 0.084(9) | 0.065(8)  | 0.062(8)  | 0.069(9)  | 0.062(9)  | 0.062(9)  | 0.063(9)  | 0.070(10) |
|                                                                                                        | 5.0 | 0.084(9) | 0.065(8)  | 0.062(8)  | 0.069(9)  | 0.062(9)  | 0.062(9)  | 0.063(9)  | 0.070(10) |
|                                                                                                        | 7.0 | 0.084(9) | 0.065(8)  | 0.062(8)  | 0.069(9)  | 0.062(9)  | 0.062(9)  | 0.063(9)  | 0.070(10) |
|                                                                                                        | 9.5 | 0.084(9) | 0.065(8)  | 0.062(8)  | 0.069(9)  | 0.062(9)  | 0.062(9)  | 0.063(9)  | 0.070(10) |
| $\theta = 0.48$<br>   | 0.0 | 0.094(9) | 0.071(8)  | 0.060(8)  | 0.054(8)  | 0.061(9)  | 0.063(9)  | 0.064(9)  | 0.066(10) |
|                                                                                                        | 0.4 | 0.091(9) | 0.068(8)  | 0.057(8)  | 0.057(8)  | 0.059(9)  | 0.060(9)  | 0.061(9)  | 0.066(10) |
|                                                                                                        | 0.8 | 0.087(9) | 0.066(8)  | 0.059(8)  | 0.058(8)  | 0.059(9)  | 0.060(9)  | 0.061(9)  | 0.068(10) |
|                                                                                                        | 1.2 | 0.077(8) | 0.068(8)  | 0.059(8)  | 0.073(9)  | 0.063(9)  | 0.072(10) | 0.089(11) | 0.091(11) |
|                                                                                                        | 1.6 | 0.076(8) | 0.068(8)  | 0.059(8)  | 0.075(9)  | 0.063(9)  | 0.068(9)  | 0.083(11) | 0.092(11) |
|                                                                                                        | 2.0 | 0.076(8) | 0.068(8)  | 0.059(8)  | 0.075(9)  | 0.063(9)  | 0.067(9)  | 0.085(10) | 0.087(11) |
|                                                                                                        | 2.4 | 0.075(8) | 0.068(8)  | 0.059(8)  | 0.076(9)  | 0.063(9)  | 0.067(9)  | 0.084(10) | 0.086(11) |
|                                                                                                        | 2.8 | 0.075(8) | 0.068(8)  | 0.059(8)  | 0.076(9)  | 0.063(9)  | 0.067(9)  | 0.084(10) | 0.085(11) |
|                                                                                                        | 3.5 | 0.075(8) | 0.069(8)  | 0.059(8)  | 0.076(9)  | 0.062(9)  | 0.067(9)  | 0.083(10) | 0.079(10) |
|                                                                                                        | 5.0 | 0.075(8) | 0.069(8)  | 0.059(8)  | 0.076(9)  | 0.062(9)  | 0.067(9)  | 0.083(10) | 0.079(10) |
|                                                                                                        | 7.0 | 0.075(8) | 0.069(8)  | 0.059(8)  | 0.076(9)  | 0.062(9)  | 0.067(9)  | 0.083(10) | 0.079(10) |
|                                                                                                        | 9.5 | 0.075(8) | 0.069(8)  | 0.059(8)  | 0.076(9)  | 0.062(9)  | 0.067(9)  | 0.083(10) | 0.079(10) |
| $\theta = 0.72$<br> | 0.0 | 0.094(9) | 0.071(8)  | 0.060(8)  | 0.054(8)  | 0.061(9)  | 0.063(9)  | 0.064(9)  | 0.066(10) |
|                                                                                                        | 0.4 | 0.054(7) | 0.155(12) | 0.112(10) | 0.165(13) | 0.150(13) | 0.196(15) | 0.189(15) | 0.192(16) |
|                                                                                                        | 0.8 | 0.047(7) | 0.151(12) | 0.107(10) | 0.157(13) | 0.153(13) | 0.197(15) | 0.172(14) | 0.181(15) |
|                                                                                                        | 1.2 | 0.052(7) | 0.090(9)  | 0.108(10) | 0.123(11) | 0.136(12) | 0.143(13) | 0.169(14) | 0.195(16) |
|                                                                                                        | 1.6 | 0.054(7) | 0.081(9)  | 0.110(10) | 0.136(12) | 0.132(12) | 0.142(13) | 0.160(14) | 0.186(15) |
|                                                                                                        | 2.0 | 0.054(7) | 0.080(9)  | 0.109(10) | 0.126(11) | 0.133(12) | 0.158(13) | 0.168(14) | 0.218(17) |
|                                                                                                        | 2.4 | 0.054(7) | 0.079(9)  | 0.110(10) | 0.128(12) | 0.137(12) | 0.160(14) | 0.170(14) | 0.223(17) |
|                                                                                                        | 2.8 | 0.054(7) | 0.079(9)  | 0.111(10) | 0.135(12) | 0.139(12) | 0.166(14) | 0.171(14) | 0.230(18) |
|                                                                                                        | 3.5 | 0.054(7) | 0.079(9)  | 0.112(10) | 0.140(12) | 0.143(12) | 0.172(14) | 0.171(14) | 0.233(18) |
|                                                                                                        | 5.0 | 0.054(7) | 0.079(9)  | 0.112(10) | 0.142(12) | 0.144(12) | 0.174(14) | 0.172(14) | 0.197(16) |
|                                                                                                        | 7.0 | 0.054(7) | 0.079(9)  | 0.112(10) | 0.142(12) | 0.144(12) | 0.175(14) | 0.172(14) | 0.197(16) |
|                                                                                                        | 9.5 | 0.054(7) | 0.079(9)  | 0.112(10) | 0.142(12) | 0.144(12) | 0.175(14) | 0.172(14) | 0.197(16) |

In each table the pattern of diffusion coefficient  $\mu$  and the strong threshold  $\theta$  (shown on the left) is fixed. The background color depends on  $\rho$  with colormap . It is scaled to range between the minimum and maximum value (among all the tables) of  $\rho$ . Strides pattern for  $\mu$  as in Fig. 1(c)

### 5.3 Relation Between $\theta$ and Computational Costs

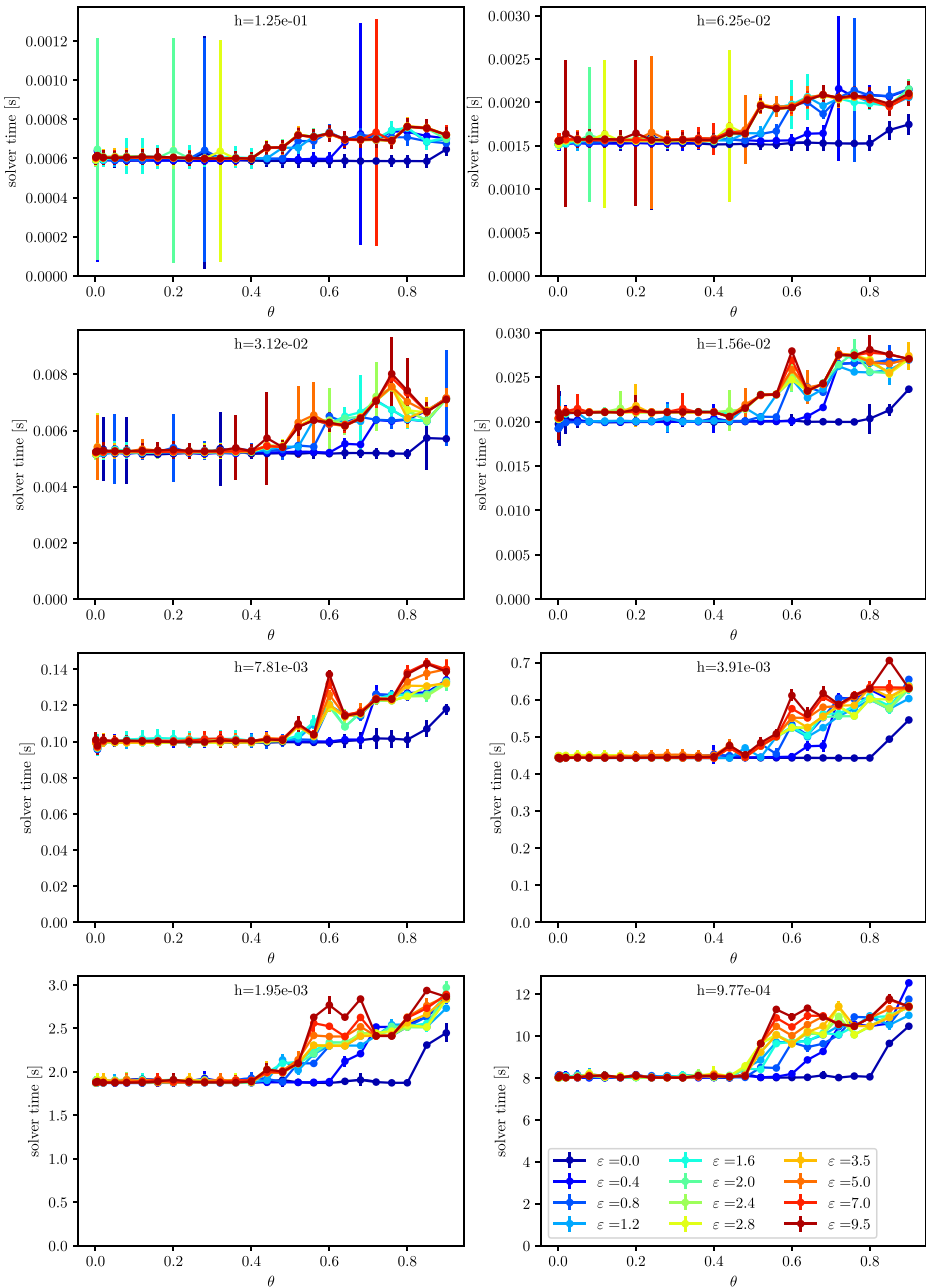
We also investigate the relation between  $\theta$  and the CPU time  $t$  needed to solve the linear system. Indeed, this is the quantity that we want to minimize in practice.

**Table 3** Computed values of the approximate convergence factor  $\rho$  and corresponding preconditioned CG iteration counts (between parenthesis) w.r.t. parameters  $\varepsilon$  on rows and mesh size  $h$  on columns

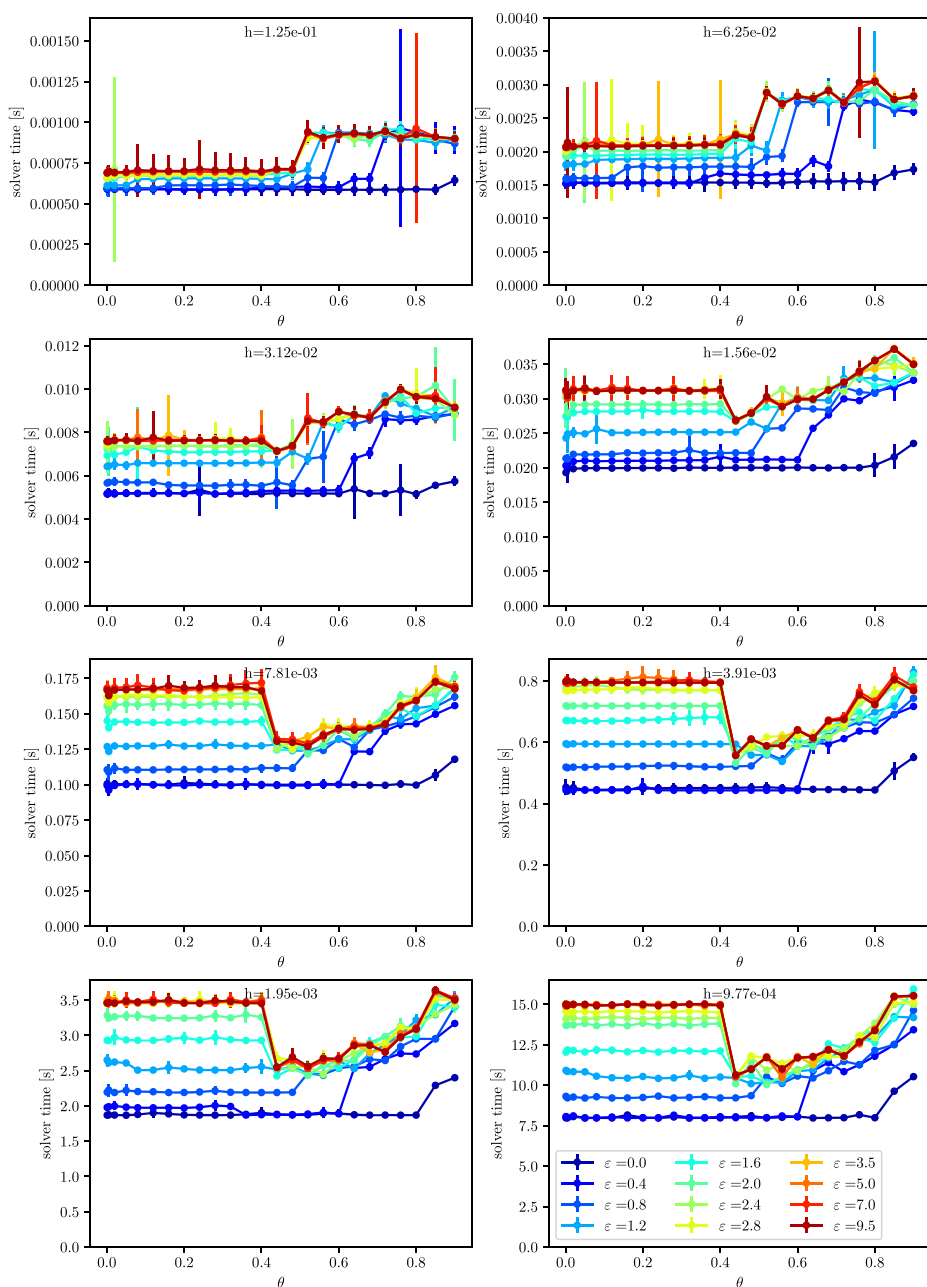
| $\varepsilon \backslash h$ |     | 1.25e-01  | 6.25e-02  | 3.12e-02  | 1.56e-02  | 7.81e-03  | 3.91e-03  | 1.95e-03  | 9.77e-04  |
|----------------------------|-----|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| $\theta = 0.24$            | 0.0 | 0.094(9)  | 0.071(8)  | 0.060(8)  | 0.054(8)  | 0.061(9)  | 0.063(9)  | 0.064(9)  | 0.066(10) |
|                            | 0.4 | 0.088(9)  | 0.070(8)  | 0.064(8)  | 0.067(9)  | 0.065(9)  | 0.064(9)  | 0.074(10) | 0.075(10) |
|                            | 0.8 | 0.097(9)  | 0.103(10) | 0.088(9)  | 0.101(10) | 0.113(11) | 0.125(12) | 0.127(12) | 0.134(13) |
|                            | 1.2 | 0.142(11) | 0.160(12) | 0.169(13) | 0.171(13) | 0.182(14) | 0.194(15) | 0.193(15) | 0.205(16) |
|                            | 1.6 | 0.166(12) | 0.196(13) | 0.220(15) | 0.228(16) | 0.240(17) | 0.255(18) | 0.268(19) | 0.277(20) |
|                            | 2.0 | 0.176(12) | 0.221(14) | 0.247(16) | 0.261(17) | 0.288(19) | 0.302(20) | 0.326(22) | 0.344(24) |
|                            | 2.4 | 0.180(12) | 0.234(15) | 0.254(16) | 0.289(19) | 0.307(20) | 0.326(22) | 0.343(24) | 0.362(25) |
|                            | 2.8 | 0.182(12) | 0.236(15) | 0.273(17) | 0.294(19) | 0.312(20) | 0.333(22) | 0.350(24) | 0.375(26) |
|                            | 3.5 | 0.196(13) | 0.237(15) | 0.275(17) | 0.297(19) | 0.317(21) | 0.333(23) | 0.355(24) | 0.386(27) |
|                            | 5.0 | 0.196(13) | 0.238(15) | 0.275(17) | 0.298(19) | 0.318(21) | 0.333(23) | 0.356(24) | 0.388(27) |
|                            | 7.0 | 0.196(13) | 0.238(15) | 0.275(17) | 0.298(19) | 0.318(21) | 0.333(23) | 0.356(24) | 0.388(27) |
|                            | 9.5 | 0.196(13) | 0.238(15) | 0.275(17) | 0.298(19) | 0.318(21) | 0.333(23) | 0.356(24) | 0.388(27) |
| $\theta = 0.48$            | 0.0 | 0.094(9)  | 0.071(8)  | 0.060(8)  | 0.054(8)  | 0.061(9)  | 0.063(9)  | 0.064(9)  | 0.066(10) |
|                            | 0.4 | 0.088(9)  | 0.077(9)  | 0.064(8)  | 0.066(9)  | 0.065(9)  | 0.063(9)  | 0.063(9)  | 0.071(10) |
|                            | 0.8 | 0.097(9)  | 0.103(10) | 0.088(9)  | 0.101(10) | 0.113(11) | 0.125(12) | 0.127(12) | 0.134(13) |
|                            | 1.2 | 0.113(10) | 0.161(12) | 0.107(10) | 0.117(11) | 0.137(12) | 0.158(13) | 0.184(15) | 0.180(15) |
|                            | 1.6 | 0.129(10) | 0.192(13) | 0.118(11) | 0.132(12) | 0.140(12) | 0.166(14) | 0.191(15) | 0.205(16) |
|                            | 2.0 | 0.147(11) | 0.210(14) | 0.124(11) | 0.126(12) | 0.152(13) | 0.169(14) | 0.194(15) | 0.225(18) |
|                            | 2.4 | 0.150(11) | 0.216(14) | 0.127(11) | 0.141(12) | 0.155(13) | 0.170(14) | 0.184(15) | 0.252(19) |
|                            | 2.8 | 0.152(11) | 0.218(14) | 0.125(11) | 0.140(12) | 0.144(12) | 0.171(14) | 0.196(16) | 0.252(19) |
|                            | 3.5 | 0.153(11) | 0.220(14) | 0.126(11) | 0.138(12) | 0.152(13) | 0.196(15) | 0.203(16) | 0.219(17) |
|                            | 5.0 | 0.153(11) | 0.220(14) | 0.126(11) | 0.138(12) | 0.153(13) | 0.196(15) | 0.211(16) | 0.225(17) |
|                            | 7.0 | 0.153(11) | 0.220(14) | 0.126(11) | 0.138(12) | 0.153(13) | 0.196(15) | 0.211(16) | 0.225(17) |
|                            | 9.5 | 0.153(11) | 0.220(14) | 0.126(11) | 0.138(12) | 0.153(13) | 0.196(15) | 0.211(16) | 0.225(17) |
| $\theta = 0.72$            | 0.0 | 0.094(9)  | 0.071(8)  | 0.060(8)  | 0.054(8)  | 0.061(9)  | 0.063(9)  | 0.064(9)  | 0.066(10) |
|                            | 0.4 | 0.039(7)  | 0.141(11) | 0.143(12) | 0.172(13) | 0.183(14) | 0.195(15) | 0.208(16) | 0.215(17) |
|                            | 0.8 | 0.045(7)  | 0.119(10) | 0.131(11) | 0.173(13) | 0.190(14) | 0.213(16) | 0.212(16) | 0.240(18) |
|                            | 1.2 | 0.052(7)  | 0.115(10) | 0.175(13) | 0.159(13) | 0.202(15) | 0.240(17) | 0.243(18) | 0.266(20) |
|                            | 1.6 | 0.057(8)  | 0.108(10) | 0.152(12) | 0.171(13) | 0.212(15) | 0.244(17) | 0.270(19) | 0.274(20) |
|                            | 2.0 | 0.052(7)  | 0.115(10) | 0.158(12) | 0.169(13) | 0.206(15) | 0.222(16) | 0.264(19) | 0.266(19) |
|                            | 2.4 | 0.052(7)  | 0.115(10) | 0.147(12) | 0.153(12) | 0.186(14) | 0.221(16) | 0.244(18) | 0.273(20) |
|                            | 2.8 | 0.052(7)  | 0.116(10) | 0.147(12) | 0.162(13) | 0.187(14) | 0.224(16) | 0.242(18) | 0.264(19) |
|                            | 3.5 | 0.053(7)  | 0.117(10) | 0.141(12) | 0.162(13) | 0.188(14) | 0.235(17) | 0.236(17) | 0.260(19) |
|                            | 5.0 | 0.053(7)  | 0.117(10) | 0.141(12) | 0.162(13) | 0.189(14) | 0.244(17) | 0.236(17) | 0.259(19) |
|                            | 7.0 | 0.053(7)  | 0.120(10) | 0.141(12) | 0.162(13) | 0.189(14) | 0.245(17) | 0.236(17) | 0.259(19) |
|                            | 9.5 | 0.053(7)  | 0.120(10) | 0.141(12) | 0.162(13) | 0.189(14) | 0.245(17) | 0.236(17) | 0.259(19) |

In each table the pattern of diffusion coefficient  $\mu$  and the strong threshold  $\theta$  (shown on the left) is fixed. The background color depends on  $\rho$  with colormap . It is scaled to range between the minimum and maximum value (among all the tables) of  $\rho$ . Checkerboard pattern for  $\mu$  as in Fig. 1(d)

In order to have an accurate estimate of the CPU time  $t$  we gather multiple samples by repeating each simulation. The number of iterations we choose was the minimum number such that the standard deviation of  $t$  did not change significantly when increasing the num-



**Fig. 3** Mean and standard deviation (visualized as errorbar) of the elapsed CPU time  $t$  to solve the linear system of equations (in seconds) based on employing the AMG preconditioned CG. In each plot we have fixed a different mesh size  $h = 1.25e-1, \dots, 9.773e-4$ . Each line represents the solver CPU time  $t$  for a fixed choice of  $\varepsilon$  entering in the definition of the diffusion coefficient  $\mu$ . The pattern of  $\mu$  is reported in Fig. 1(c)



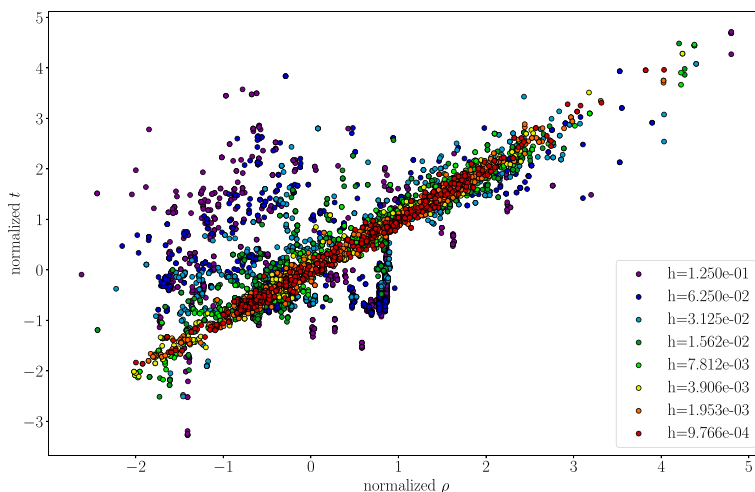
**Fig. 4** Mean and standard deviation (visualized as errorbar) of the elapsed CPU time  $t$  to solve the linear system of equations (in seconds) based on employing the AMG preconditioned CG. In each plot we have fixed a different mesh size  $h = 1.25e-1, \dots, 9.773e-4$ . Each line represents the solver CPU time  $t$  for a fixed choice of  $\varepsilon$  entering in the definition of the diffusion coefficient  $\mu$ . The pattern of  $\mu$  is reported in Fig. 1(d)

ber of samples. Namely, we employed 200, 100, 50, 20, 10, 7, 5, and 4 iterations for each mesh refinement from the coarsest to the finest, respectively. We report the plots of  $t$  vs  $\rho$  in Figs. 3 and 4. Notice that despite the large number of repetitions the standard deviation (shown as an errorbar) in some cases is still large. These results are also useful to analyze the relation between  $\theta$  and  $\rho$  since we will show that the least square analysis seems to indicate that they have a linear relation. We also observe that for small values of the strong threshold parameter ( $\theta \leq 0.3$ ) there is an interval where the CPU time is almost constant: this appears to be true for all the test cases addressed. Since a smaller strong threshold parameter means that more connections are discarded in the coarsening phase, one would expect that as  $\theta$  gets smaller, then the approximate convergence factor  $\rho$  may deteriorate, which in turn would lead to larger CPU times. A possible motivation of this behaviour is to consider that among the settings of BoomerAMG there is parameter that prevents the coarsening from being too small (in the present test its value has been set as default, i.e. equal to 1). Thus, the coarse system associated to  $A_H$  is still effectively damping the smooth components. Indeed, from Fig. 2 we can see how the number of levels and the size of the coarse system is constant for small  $\theta$ .

### 5.3.1 Choice of the Performance Index $p$

We are now interested in finding a scalar  $p$  that evaluates how good the AMG configuration is. Two possible choices for such performance index are the approximate convergence factor  $\rho$ , which measures how rapidly the linear solver converges, and the elapsed CPU time  $t$ .

We now proceed to analyze the relation between the elapsed CPU time  $t$  and the approximate convergence factor  $\rho$ . In Fig. 5, we show a scattered plot of the elapsed CPU time ( $t$ ) as a function of  $\rho$ , for different values of the mesh size  $h$ . The results are normalized with respect to the data that belong to the same test case. A linear relation between  $t$  and  $\rho$  can be clearly identified. This is also confirmed by the results shown in Table 4, where we report the least square analysis of the data of Fig. 5. We highlight that these results support



**Fig. 5** Scatter plot of the (average) elapsed CPU time ( $t$ ) versus the approximate convergence factor  $\rho$ . Different colors identify different mesh sizes  $h$ . The data are normalized (in both components) with respect to the corresponding data in the same test case

**Table 4** Linear least square analysis of the model  $t = \hat{\rho}_1 \rho + \hat{\rho}_0$ 

| $h$                    | 1.25e-1 | 6.25e-2 | 3.12e-2 | 1.56e-2  | 7.81e-3 | 3.91e-3 | 1.95e-3 | 9.77e-4 |
|------------------------|---------|---------|---------|----------|---------|---------|---------|---------|
| $nn$                   | 1200    | 1200    | 1200    | 1200     | 1200    | 1200    | 1200    | 1200    |
| $R^2$                  | 0.155   | 0.564   | 0.762   | 0.912    | 0.985   | 0.992   | 0.991   | 0.993   |
| F-statistic            | 220.5   | 1550    | 3836    | 1.24e4   | 7.88e4  | 1.58e5  | 1.34e5  | 1.68e5  |
| AIC                    | -2.15e4 | -1.79e4 | -1.53e4 | -1.30e+4 | -1.09e4 | -7864   | -4048   | -800.1  |
| $\hat{\rho}_1$         | 3.49e-4 | 3.05e-3 | 1.25e-2 | 4.99e-2  | 0.263   | 1.245   | 5.259   | 20.95   |
| $SE(\hat{\rho}_1)$     | 2.35e-5 | 7.76e-5 | 2.02e-4 | 4.48e-4  | 9.35e-4 | 3.13e-3 | 1.42e-2 | 5.11e-2 |
| t-value $\hat{\rho}_1$ | 14.848  | 39.369  | 61.937  | 111.299  | 280.677 | 397.857 | 366.596 | 409.565 |
| p-value $\hat{\rho}_1$ | < 0.001 | < 0.001 | < 0.001 | < 0.001  | < 0.001 | < 0.001 | < 0.001 | < 0.001 |

Data are grouped by the mesh size  $h$ .  $t$  is the elapsed CPU time and  $\rho$  is the approximate converge factor. For the definition of the quantities appearing in the first column we refer to Table 1

the hypothesis of a relation between  $\rho$  and  $t$ . We can explain the poor correlation for coarse mesh sizes  $h$  due to the higher relative uncertainty of the measure. Indeed, as  $h$  gets smaller the coefficient of determination  $R^2$  improves.

Therefore, in the following we use as performance index the convergence factor  $\rho$ . Indeed,  $\rho$  is not machine nor implementation dependent, thus leading to reproducible results.

The dataset that we build contains numerical simulations made with every combination of parameters among 8 mesh sizes  $h$ , 25 values of  $\theta$ , 12 values of  $\varepsilon$  and 4 patterns of  $\mu$  (Fig. 1) for a total of 9600 samples.

## 6 ANN-Enhanced AMG Method

In this section, we design and use ANNs to predict the value of the strong threshold parameter  $\theta^*$  that maximizes the performance of the AMG method, measured in terms of corresponding convergence factor  $\rho$ . Our goal is to design a model, namely an ANN, that enables predictions of the optimal strong threshold  $\theta$  for a given model problem. We remark that, in the framework discussed in Section 3, fixing a test case (model problem) is equivalent to fixing the matrix  $A_h$  defined in (6). We define the optimal value of strong threshold parameter  $\theta^*$  for a certain test case as the minimizers of the convergence factor  $\rho = \rho(A_h, \theta)$ .

Then, we build our model (the ANN) to predict the convergence factor  $\rho$  of the AMG in a fixed test case and with a fixed strong threshold parameter  $\theta$ . More precisely,  $\mathcal{F}$  is the ANN such that

$$\mathcal{F}(\text{normalize}(\text{pooling}(A_h, m, \text{op})), -\log_2(h), \theta; \boldsymbol{\gamma}) = \rho.$$

Here, the `pooling` will be introduced in Section 6.1.1, whereas  $m$ , `normalize` and `op` are so-called hyperparameters of the model (that will be introduced and discussed in detail in Section 6.1.2). Finally,  $\boldsymbol{\gamma}$  are the parameters that define the ANN (see (9)). There are two reasons to adopt this approach: first, it is possible to quantify the improvement on the performance that we expect; second, each numerical simulation can be added to the dataset making this process less computationally expensive and more flexible.

If otherwise not stated, we will use a 60%-20%-20% split of the dataset into training-validation-test.

## 6.1 ANN-Based Prediction of the Optimal Strong Threshold Parameter $\theta$

In this section, we discuss how to predict the optimal strong threshold parameter  $\theta$  to be used in the ANN-enhanced AMG Algorithm 4 without using any prior assumption on the diffusion coefficient  $\mu$ . In other words, we do not rely on the fact that  $\mu$  shows a finite number of patterns.

The variables that we use as inputs of the ANN  $\mathcal{F}$  are  $\theta$ ,  $-\log_2(h)$  and a set of variables  $\hat{V} = \text{normalize}(\text{pooling}(A_h, m, \text{op}))$  that is extracted from the matrix  $A_h$  of the linear system (6) by means of the pooling Algorithm 3 and a normalization algorithm (extraction step). This extraction process should be computationally cheap, indeed this approach is worthwhile only until the process of predicting the optimal value of  $\theta$  has a negligible computational cost with respect to the elapsed CPU time to solve the linear system. We use  $-\log_2(h)$  instead of  $h$  as input of the ANN since, by construction,  $h$  is not linearly distributed. It is a well known that ANNs can be more easily trained if the inputs have the same order of magnitude.

We recall at this stage that the optimal parameter  $\theta^*$  to be used in the linear solver with AMG preconditioner (step 4 of Algorithm 4) is such that

$$\theta^* = \underset{\theta \in (0,1]}{\operatorname{argmin}} \mathcal{F}(\hat{V}, -\log_2(h), \theta; \boldsymbol{\gamma}).$$

In practice,  $\theta^*$  is found by first evaluating

$$\theta^{(0)} = \underset{k \in \{20, 30, \dots, 900\}}{\operatorname{argmin}} \mathcal{F}(\hat{V}, -\log_2(h), 0.001k; \boldsymbol{\gamma}),$$

and then applying a suitable number of steps of the gradient descent algorithm

$$\theta^{(k)} = \theta^{(k-1)} - \alpha \nabla_{\theta} \mathcal{F}(\hat{V}, -\log_2(h), \theta^{(k-1)}; \boldsymbol{\gamma}), \quad k \geq 0,$$

where  $\alpha = 10^{-5}$  is the learning rate. The gradient  $\nabla_{\theta} \mathcal{F}$  can be computed by the automatic differentiation algorithm of Tensorflow. However, we empirically found that this second step appears to be unnecessary since it gives small to negligible improvements.

### 6.1.1 Pooling (Step 1 of Algorithm 4)

We introduce what we call the *view*  $V$  of the matrix  $A_h$ .

First, let us define the following hyperparameters. Let  $m \in \mathbb{N}$  be a positive integer that describes the size of  $V \in \mathbb{R}^{m \times m}$ . It must be large enough so that the features of  $A_h$  are not lost. At the same time,  $m$  should not be too large to avoid expensive computations in the forward propagation step. Let  $\text{op} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$  be a function that combines two values. In the field of computer vision this function is usually the sum  $\text{op} = \text{sum}$  where  $\text{sum}(v_1, v_2) = v_1 + v_2$ , or the maximum of two numbers.

We define the view  $V \in \mathbb{R}^{m \times m}$  of the matrix  $A_h$  and the non-zeros count  $C \in \mathbb{N}^{m \times m}$  (i.e. the matrix where each entry  $(C)_{ij}$  is the number of non-zero elements of  $A_h$  used to compute  $(V)_{ij}$ ) as  $(V, C) = \text{pooling}(A_h, m, \text{op})$ , where  $\text{pooling}$  is defined in Algorithm 3. The insight in this algorithm stems from the operator used in the pooling layer of CNNs. On one hand we downscale the input, significantly reducing the computational cost, and on the other hand we also gain translation invariance. Moreover, we also prune details that may not be useful for the task.

By exploiting the storage data structure of sparse matrices (for instance compressed row storage or coordinate lists) to access the elements of  $A_h$ , we realize the pooling with complexity  $O(nnz)$ , where  $nnz$  is the number of non-zero elements in the matrix  $A_h$ . In Algorithm 3, for the sake of simplicity, it is assumed that the matrix  $A_h$  is stored in coordinate lists format.

---

```

1 access  $A_h$  in COO form and extract its size:  $val, row, col, n \leftarrow A_h$ ;
2 initialize  $V$  to an  $m \times m$  dense matrix with all zero entries;
3 initialize  $C$  to an  $m \times m$  dense matrix with all zero entries;
4  $q \leftarrow n/m$ ;
5  $p \leftarrow n \bmod m$ ;
6  $t \leftarrow (q + 1)p$ ;
7 for  $k = 0$  to  $val.size() - 1$  do
8    $i \leftarrow row[k]/(q + 1)$  if  $(row[k] < t)$  else  $(row[k] - t)/q + p$ ;
9    $j \leftarrow col[k]/(q + 1)$  if  $(col[k] < t)$  else  $(col[k] - t)/q + p$ ;
10   $V_{ij} \leftarrow op(V_{ij}, val[k])$ ;
11   $C_{ij} \leftarrow C_{ij} + 1$ ;
12 end
13 return  $V, C$ ;
```

---

**Algorithm 3** Pooling algorithm  $(V, C) = \text{pooling}(A_h, m, op)$ .

We have also measured the elapsed CPU time by the pooling algorithm. In each simulation it seems that Algorithm 3 requires a negligible CPU time compared to the global one. We notice that this algorithm could easily be extended to work in parallel. We also point out that Algorithm 3 does not rely on the connectivity of the mesh nor on the definition of the coefficients, thus it should also work in more complex cases, as for example in the case of unstructured meshes.

### 6.1.2 Normalization (Step 2 in Algorithm 4)

We observe that the *view*  $V$  defined in the previous section cannot be used as input of an ANN yet. In particular, it features very large values that might impact the stability of the gradient algorithm. For this reason, propose three normalization techniques:

$$\begin{aligned}
 \text{std}(V)_{ij} &= \frac{(V)_{ij} - \bar{v}}{\sigma}, \quad \bar{v} = \sum_{i,j} \frac{(V)_{ij}}{m^2}, \quad \sigma = \sqrt{\frac{1}{m^2} \sum_{i,j} [(V)_{ij} - \bar{v}]^2}, \\
 \text{scale}(V)_{ij} &= (V)_{ij} / \max_{i,j} |(V)_{ij}|, \\
 \log(V)_{ij} &= \text{scale}(\log(|(V)_{ij}| + 1)(V)_{ij} / |(V)_{ij}|).
 \end{aligned} \tag{11}$$

The first approach is the most employed in the field of deep learning. The argument behind the second and third definition of (11) is that we would like to preserve the sparsity pattern of the matrix. In particular, the  $\log$  normalization yields linearly distributed values, since the exponent  $\varepsilon$  of diffusion coefficient  $\mu = 10^\varepsilon$  is linearly distributed. Another possibility is to apply these normalization to the element-wise division of  $V$  and  $C$

$$\text{avg}(V, C)_{ij} = \frac{(V)_{ij}}{(C)_{ij}}, \tag{12}$$

**Table 5** Summary of the normalization techniques for the view  $V$ 

| Name      | normalize                                  |
|-----------|--------------------------------------------|
| std+id    | $\hat{V} = \text{std}(V)$                  |
| std+avg   | $\hat{V} = \text{std}(\text{avg}(V, C))$   |
| scale+id  | $\hat{V} = \text{scale}(V)$                |
| scale+avg | $\hat{V} = \text{scale}(\text{avg}(V, C))$ |
| log+id    | $\hat{V} = \log(V)$                        |
| log+avg   | $\hat{V} = \log(\text{avg}(V, C))$         |

See (11) and (12) for further details

with the exception that if  $(C)_{ij} = 0$  then  $\text{avg}(V, C)_{ij} = 0$ . Hence, the sparsity pattern of  $V$  is preserved, indeed  $(C)_{ij} = 0$  only if  $(V)_{ij} = 0$ . Table 5 summarizes all the normalization techniques we propose. Figure 6 shows an example of the sparsity pattern of the matrix  $A_h$  ( $h = 1.25e-1$ ) and Fig. 7 shows three examples of a side-to-side view of the four normalizations  $\hat{V}$  of the view matrix  $V$ . At this stage, we have obtained a matrix  $\hat{V} \in \mathbb{R}^{m \times m}$ ,  $\hat{V} = \text{normalize}(V, C)$  with  $m$  chosen a priori. We will discuss our choice in the next section.

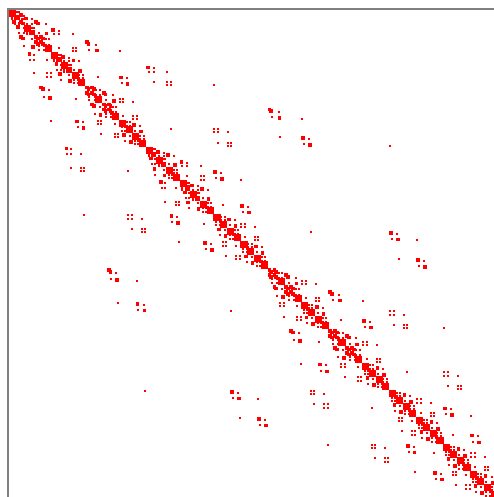
- 
- 1  $V, C \leftarrow \text{pooling}(A_h, m, \text{op})$  (Algorithm 3);
  - 2  $\hat{V} \leftarrow \text{normalize}(V, C)$  (Table 5);
  - 3  $\theta^* \leftarrow \text{argmin}_{\theta} \mathcal{F}(\hat{V}, -\log_2(h), \theta; \gamma)$ ;
  - 4  $\mathbf{u}_h^{(k+1)} \leftarrow \text{AMG}(\mathbf{u}_h^{(0)}, A_h, \mathbf{f}_h, \theta^*, \nu_1, \nu_2, N_{\max}, \text{tol})$  (Algorithm 2)
- 

**Algorithm 4** ANN-enhanced AMG  $\mathbf{u}_h^{(k+1)} = \text{ANN\_AMG}(\mathbf{u}_h^{(0)}, A_h, h, \mathbf{f}_h, \nu_1, \nu_2, N_{\max}, \text{tol}, \gamma)$ .

### 6.1.3 The ANN-Enhanced AMG Algorithm

We show in Algorithm 4 how we intend to use the prediction of the optimal strong threshold parameter  $\theta^*$  realized by ANN within the AMG solver, which we call ANN-enhanced AMG algorithm. In particular, our approach determines  $\theta^*$  to be used in the AMG algorithm starting from the matrix  $A_h$  and the mesh size  $h$ . This leverages on a map from a manipulation of  $A_h$  ( $\hat{V}$ ),  $h$  and  $\theta$  to a suitable performance index  $p$  of the AMG solver ( $\rho$ ). Specifically, this map is realized by an ANN  $\mathcal{F}(\mathbf{x}; \gamma)$  such that its inputs are  $\mathbf{x} = (\hat{V}, -\log_2(h), \theta)$ , while the output  $\mathbf{y}$  (the predicted value of the regression) coincides with a suitable performance index, say  $\mathbf{y} = p(A_h, h, \theta)$ , of the linear solver with AMG preconditioner, which we select as the approximated convergence factor  $p = \rho(A_h, \theta)$ . The steps in the ANN-enhanced AMG Algorithm 4 are the following:

- (1–2) as the matrix  $A_h$  can not be directly used as input of an ANN  $\mathcal{F}$ , suitable pooling and normalization steps are performed to assemble  $\hat{V} = \text{normalize}(\text{pooling}(A_h, m, \text{op}))$  from  $A_h$ , where `pooling` is defined in Algorithm 3, and `op`, `normalize` and  $m$  are hyperparameters of our model defined inside Section 6.1.1 and Section 6.1.2, respectively;
- (3) the ANN  $\mathcal{F}$  built for the model problem is used to determine  $\theta^*$  in order to minimize the approximate convergence factor  $\rho$  of the AMG;
- (4) the AMG Algorithm 2 is used with  $\theta^*$ .



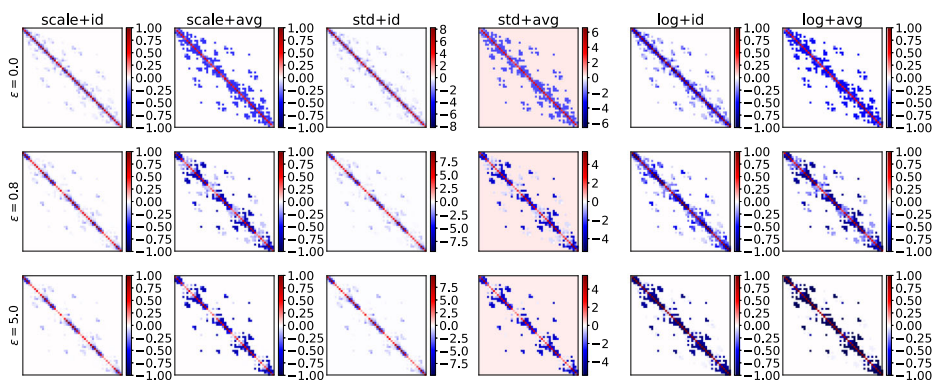
**Fig. 6** Sparsity pattern of the matrix  $A_h$  for  $h = 1.25\text{e-}1$  of the case 1 (Section 6.3)

#### 6.1.4 ANN Architecture (Building the ANN of Step 3 in Algorithm 4)

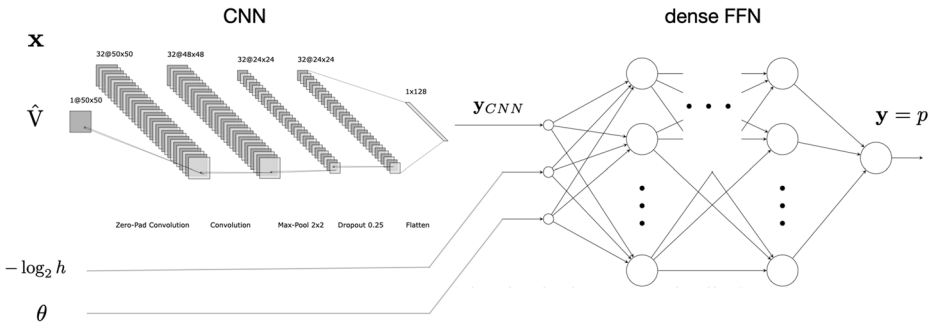
We now build the ANN  $\mathcal{F}(\mathbf{x}; \boldsymbol{\gamma}) = \mathcal{F}(\hat{\mathbf{V}}, -\log_2(h), \theta; \boldsymbol{\gamma})$ . We recall that, following the former pooling and normalization steps, we have

$$\hat{\mathbf{V}} = \text{normalize}(\text{pooling}(A_h, m, \text{op})),$$

where `pooling` is defined in Algorithm 3 and `op`, `normalize` and  $m$  are the hyperparameters defined in Section 6.1.1 and Section 6.1.2, respectively. We use a model that is the



**Fig. 7** Graphical representation of six different normalization of the view matrix  $\hat{\mathbf{V}} = \text{normalize}(\text{pooling}(A_h, 50, \text{sum})) \in \mathbb{R}^{50 \times 50}$  for mesh dimension  $h = 1.25\text{e-}1$ ,  $\mu$  pattern Fig. 1(d) and  $\varepsilon = 0, 0.8, 5$  for the first, second and third rows, respectively



**Fig. 8** Architecture of our model represented by the ANN  $\mathcal{F}(\hat{V}, -\log_2(h), \theta; \gamma) = p = \rho$  where  $\hat{V} = \text{normalize}(\text{pooling}(A_h, 50, \text{sum}))$ . In particular, it is comprised by the composition of two ANNs: a CNN such that  $y_{CNN} = \mathcal{F}_{CNN}(\hat{V}; \gamma_{CNN})$  and a dense FFN  $\mathcal{F}_{FFN}(y_{CNN}, -\log_2(h), \theta; \gamma_{FFN}) = \rho$ . Given  $\hat{V}$ , the mesh dimension  $h$  and  $\theta$  it predicts  $\rho$  and thus also the optimal  $\theta^*$  that minimizes  $\rho$

composition of two networks as depicted in Fig. 8. Since the matrix view  $\hat{V}$  is a structured input, we first employ a CNN such that

$$y_{CNN} = \mathcal{F}_{CNN}(\hat{V}; \gamma_{CNN}),$$

depending on the parameters  $\gamma_{CNN}$ . Then, the output of the first CNN altogether with the remaining inputs  $-\log_2(h)$  and  $\theta$ , constitute the inputs of a second network, which we select as a dense Feed Forward Network (FFN). This dense FFN is such that

$$\rho(A_h, \theta) = \mathcal{F}_{FFN}(y_{CNN}, -\log_2(h), \theta; \gamma_{FFN})$$

and depends on the parameters  $\gamma_{CNN}$ .

### 6.1.5 Evaluating the Performance of the Model

Since the a priori choice of the strong threshold parameter  $\theta$  is based on the map  $A_h \rightarrow \theta^*$  defined by step 3 of Algorithm 4, it is not enough to have a small loss to verify that the model is accurate. With this aim, we introduce some quantities of interest. Let  $A_h$  be fixed, and let

- $\rho_{\text{ANN}}$  be the convergence factor of the AMG-ANN algorithm,
- $\rho_{0.25}$  be the convergence factor of the AMG method for  $\theta = 0.25$ ,
- $\rho_{\text{MIN}}$  be the convergence factor of the AMG method with

$$\theta^* = \underset{\theta \in \text{dataset for this } A_h}{\operatorname{argmin}} \quad \rho(\theta; A_h).$$

Moreover, we define

$$P = 1 - \frac{\rho_{\text{ANN}}}{\rho_{0.25}} \quad \text{and} \quad P_{\text{MAX}} = 1 - \frac{\rho_{\text{MIN}}}{\rho_{0.25}}. \quad (13)$$

Finally, we define  $PB$  as the percentage of cases where  $P \geq 0$ , and we define  $P_{<0}$  as the performance  $P$  of the cases where  $P < 0$ .

**Table 6** Computed loss (MSE) and MAE for different ANNs architectures trained with dataset 1

| $W_1$ | $D_1$ | $P_1$ | $W_2$ | $D_2$ | $P_2$ | $O$ | $W_3$ | $D_3$ | loss    | MAE     |
|-------|-------|-------|-------|-------|-------|-----|-------|-------|---------|---------|
| 32    | 2     | 0.25  | —     | —     | —     | 128 | 64    | 2     | 7.36e-5 | 4.33e-3 |
| 32    | 2     | 0.25  | 32    | 2     | 0.5   | 128 | 64    | 2     | 9.28e-5 | 5.40e-3 |
| 32    | 2     | 0.0   | —     | —     | —     | 128 | 64    | 3     | 7.85e-5 | 4.96e-3 |
| 32    | 2     | 0.25  | —     | —     | —     | 128 | 64    | 3     | 7.72e-5 | 4.94e-3 |
| 32    | 2     | 0.5   | —     | —     | —     | 128 | 64    | 3     | 7.86e-5 | 5.16e-3 |
| 32    | 2     | 0.25  | —     | —     | —     | 256 | 64    | 3     | 8.19e-5 | 5.13e-3 |
| 32    | 2     | 0.25  | 64    | 2     | 0.5   | 128 | 64    | 4     | 1.88e-4 | 9.68e-3 |

The quantities  $W_1$ ,  $D_1$ ,  $P_1$ ,  $W_2$ ,  $D_2$ ,  $P_2$ ,  $O$ ,  $W_3$  and  $D_3$  are defined in Section 6.1.4. The batch size is 32, normalization std+id (see Table 5), training lasts 500 epochs and the optimizer is the Adam algorithm (with default Tensorflow learning rate)

## 6.2 Test Case 1

We test our algorithm fixing the hyperparameters of the view, we discuss their tuning in Section 6.4.1. Namely, we employ  $\text{op}=\text{sum}$ ,  $m = 50$  (our choice is motivated by interpreting  $\hat{V}$  as a color image in input to the first CNN network; experience indicates that this kind of CNN network is able to excellently process color images of similar size) and  $\text{normalize}=\text{std}+\text{id}$  (see Table 5).

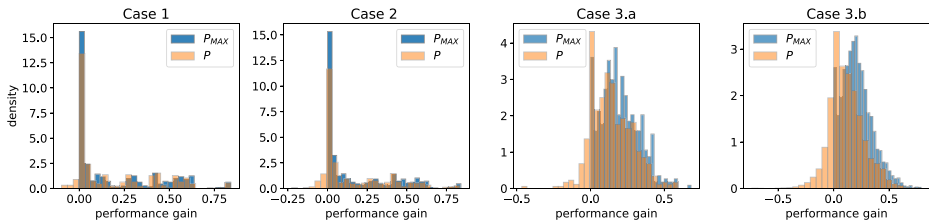
Table 6 shows the results of changing the architecture of the model. In particular, we consider architectures with two convolutional layers each composed by a convolution with zero-padding,  $3 \times 3$  kernel and ReLU activation and  $D_i - 1$  other convolutions with  $3 \times 3$  kernel and ReLU activation (without padding). The last elements of the convolutional layers are a  $2 \times 2$  max-pooling and Dropout with rate  $P_i$ , each layer has  $W_i$  hidden units (for  $i = 1, 2$ ). The output of the convolutional part has  $O$  hidden units; the dense part is composed by  $D_3$  dense layers with  $W_3$  hidden units.

In Table 7 we report the performance indexes for the first six models of Table 6 (see Section 6.1.5). We have chosen as architecture for our model the one reported in the first row of Table 6. We trained this model for up to 1000 epochs and employed early stopping. As result, it has a loss of  $6.36 \cdot 10^{-5}$ . We show the computed performance in Table 16 (first row). Figure 9 (left) shows a histogram of the performance gain  $P$ . We observe that in 20% of the cases we have a performance gain  $P \geq 43\%$ .

**Table 7** Evaluation of the performance of the first six models of Table 6

| $PB$   | $P$ (avg/median) |        | $P/P_{\text{MAX}}$ (avg/median) |        | $P_{<0}$ (avg/median) |          |
|--------|------------------|--------|---------------------------------|--------|-----------------------|----------|
| 92.96% | 16.06%           | 24.69% | 81.31%                          | 97.51% | − 3.928%              | − 3.090% |
| 90.36% | 16.63%           | 33.74% | 85.95%                          | 97.14% | − 4.102%              | − 2.123% |
| 92.70% | 16.64%           | 20.72% | 81.18%                          | 98.80% | − 2.363%              | − 0.805% |
| 92.96% | 16.56%           | 17.97% | 80.09%                          | 97.56% | − 1.855%              | − 0.826% |
| 91.66% | 16.31%           | 22.28% | 82.39%                          | 99.08% | − 3.462%              | − 1.324% |
| 91.14% | 15.59%           | 22.32% | 80.87%                          | 96.80% | − 3.812%              | − 1.094% |

The quantities  $PB$ ,  $P$ ,  $P_{\text{MAX}}$  and  $P_{<0}$  are defined at the end of Section 6.1.4



**Fig. 9** Histogram of the performance of the AMG-ANN method: for each test case we evaluate the performance of the model  $P$  in orange and the best performance  $P_{MAX}$  in blue (as defined in (13)). *From left to right.* Performance for Case 1 of Section 6.3. Performance for the Case 2 of Section 6.4. Performance for the Case 3.a and 3.b of Section 6.5

### 6.3 Test Case 2: an Enhanced Dataset

In order to further test the robustness of the model to unseen data (i.e. test cases that are not in the training set), we test the prediction capabilities of the ANN on a new dataset. We call the latter dataset, “dataset 2”, while the one employed so far is called “dataset 1”. In particular, we solve the same model problem (3) but with a different diffusion coefficient, defined as

$$\mu(x, y) = \begin{cases} 10^{\varepsilon_2} & \text{if } (x, y) \in \Omega_{gray}, \\ 10^{\varepsilon_1} & \text{if } (x, y) \in \Omega_{white}, \end{cases}$$

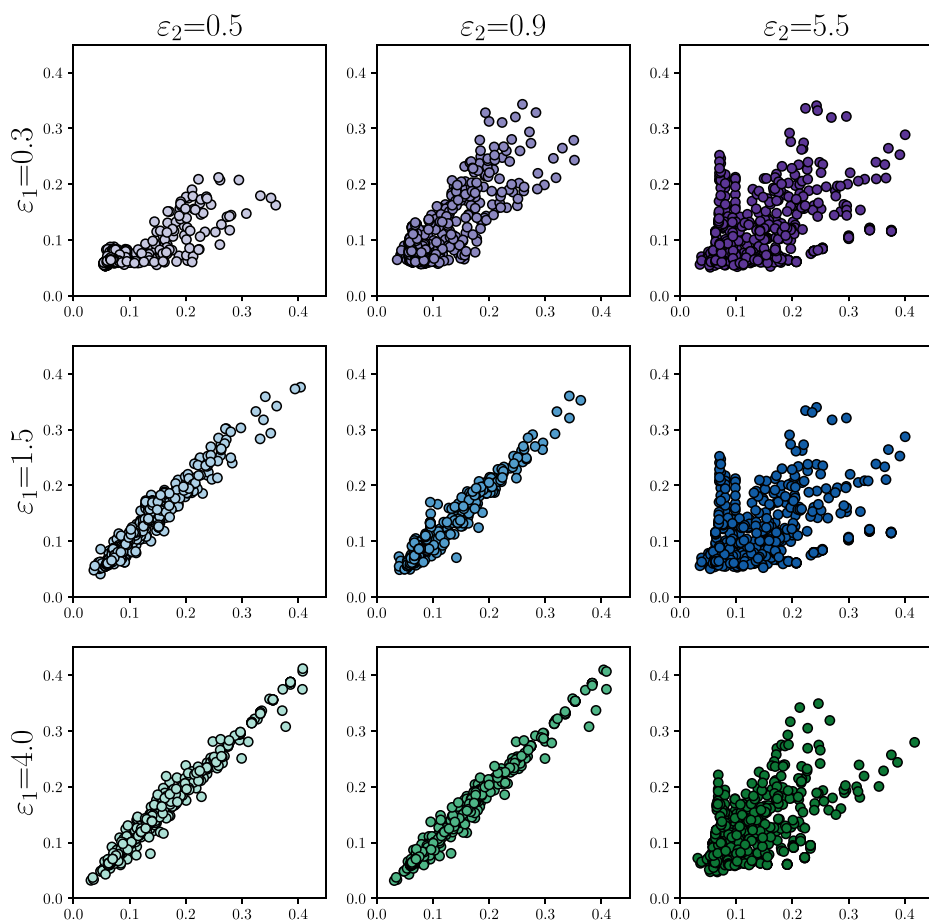
where  $\varepsilon_1$  and  $\varepsilon_2$  are parameters to be chosen and  $\Omega_{gray}, \Omega_{white}$  is a partition of  $\Omega$  as shown in Fig. 1.

In Fig. 10, we show the performance of the model that we designed in the previous sections, trained with only dataset 1. In particular, we choose randomly three values for  $\varepsilon_1$  and three values for  $\varepsilon_2$ . We can see how the predictions maintain accuracy in some scenarios, but fail in other ones.

We proceed to show how the model behaves when the training is instead done with training samples from both datasets. Dataset 2 contains 5184 entries, we define the test set to be the union of the 20% of dataset 2 and the 50% of the dataset 1. In this way, the union of the training and validation set contains 4800 datapoints from the dataset 1 and 4147 from dataset 2. The ratio between the number of entries of the validation set and the training set is defined to be 1 : 3. We call this combination dataset 3. The aim is to have a balanced training dataset in which each definition of  $\mu$  is equally represented.

If not otherwise stated, we stop the training at 200 epochs. As shown in Table 8, dropout improves the training, thus it will be employed in all the models. We have also tried employing batch normalization as a regularization technique on some of these models and a deeper model with three convolutional layers but it did not lead to any significant improvements. This can be explained by the fact that batch normalization effectiveness is most evident in very deep models; see [26].

Table 9 shows training of models with two convolutional layers. By comparing it with Table 10, where the MSE and MAE are reported for different ANN architectures with one layer, it is possible to notice that models with only one layer achieve lower loss. From Table 10, it is also possible to appreciate that the model that in the previous section achieved the lowest loss is not the same in this case. In particular, a deeper model performs better. This is not surprising since this means that we need a more complex model to explain the data, and indeed we are using a more diversified dataset. In Table 11, we repeat the same test case for different architectures of the convolutional layer. Two applications of convolution



**Fig. 10** Predictions on dataset 2 made by the model designed in Section 6.1.4 and trained with dataset 1. Each plot show data obtained from a fixed combination of  $\varepsilon_1$  and  $\varepsilon_2$ . Namely,  $\varepsilon_1$  is constant for subplots on the same row and  $\varepsilon_2$  is constant for subplots on the same column of the plot grid. On the x-axis there is the true value of  $\rho$ , on the y-axis the predicted value

**Table 8** Computed loss (MSE) and MAE for different ANNs architectures (with one convolutional layer) trained with dataset 3

| $W_1$ | $D_1$ | $P_1$ | $O$ | $W_3$ | $D_3$ | loss    | MAE     |
|-------|-------|-------|-----|-------|-------|---------|---------|
| 32    | 2     | 0.05  | 128 | 64    | 3     | 1.75e-4 | 8.60e-3 |
| 32    | 2     | 0.00  | 128 | 64    | 4     | 1.63e-4 | 8.03e-3 |
| 32    | 2     | 0.25  | 128 | 64    | 3     | 1.58e-4 | 8.14e-3 |
| 32    | 2     | 0.25  | 128 | 64    | 4     | 1.35e-4 | 7.34e-3 |
| 32    | 2     | 0.50  | 128 | 64    | 3     | 1.55e-4 | 7.98e-3 |

The quantities  $W_1$ ,  $D_1$ ,  $P_1$ ,  $O$ ,  $W_3$  and  $D_3$  are defined in Section 6.1.4. The batch size is 32, `normalize=std+id` (see Table 5), training lasts 200 epochs and the optimizer is the Adam algorithm (with default Tensorflow learning rate)

**Table 9** Computed loss (MSE) and MAE for different ANNs architectures (with two convolutional layers) trained with dataset 3

| $W_1$ | $D_1$ | $P_1$ | $W_2$ | $D_2$ | $P_2$ | $O$ | $W_3$ | $D_3$ | loss    | MAE     |
|-------|-------|-------|-------|-------|-------|-----|-------|-------|---------|---------|
| 16    | 2     | 0.25  | 12    | 2     | 0.25  | 128 | 64    | 4     | 1.72e-4 | 8.34e-3 |
| 16    | 2     | 0.25  | 12    | 2     | 0.50  | 256 | 256   | 3     | 1.56e-4 | 8.10e-3 |
| 16    | 2     | 0.25  | 16    | 2     | 0.25  | 128 | 64    | 4     | 1.68e-4 | 8.28e-3 |
| 16    | 2     | 0.25  | 32    | 2     | 0.50  | 128 | 64    | 4     | 1.61e-4 | 8.23e-3 |
| 32    | 2     | 0.25  | 16    | 2     | 0.25  | 128 | 64    | 3     | 1.55e-4 | 8.05e-3 |
| 32    | 2     | 0.25  | 16    | 2     | 0.25  | 128 | 64    | 4     | 1.55e-4 | 7.85e-3 |
| 32    | 2     | 0.25  | 32    | 2     | 0.50  | 128 | 64    | 4     | 1.75e-4 | 8.64e-3 |

The quantities  $W_1$ ,  $D_1$ ,  $P_1$ ,  $W_2$ ,  $D_2$ ,  $P_2$ ,  $O$ ,  $W_3$  and  $D_3$  are defined in Section 6.1.4. The hyperparameters are the same of Table 8

**Table 10** Computed loss (MSE) and MAE for different ANNs architectures (with one convolutional layer) trained with dataset 3

| $W_1$ | $D_1$ | $P_1$ | $O$ | $W_3$ | $D_3$ | loss    | MAE     |
|-------|-------|-------|-----|-------|-------|---------|---------|
| 32    | 2     | 0.25  | 128 | 64    | 3     | 1.71e-5 | 8.19e-3 |
| 32    | 2     | 0.25  | 128 | 64    | 4     | 1.35e-5 | 7.34e-3 |
| 32    | 2     | 0.25  | 128 | 64    | 5     | 1.48e-5 | 7.88e-3 |
| 32    | 2     | 0.25  | 128 | 128   | 3     | 1.51e-5 | 7.60e-3 |
| 32    | 2     | 0.25  | 128 | 128   | 4     | 1.43e-5 | 7.62e-3 |
| 32    | 2     | 0.25  | 256 | 64    | 5     | 1.50e-5 | 7.86e-3 |
| 32    | 2     | 0.25  | 256 | 256   | 3     | 1.52e-5 | 7.80e-3 |
| 32    | 2     | 0.25  | 512 | 128   | 3     | 1.60e-5 | 8.01e-3 |
| 32    | 3     | 0.25  | 128 | 64    | 4     | 1.51e-5 | 7.64e-3 |
| 32    | 3     | 0.25  | 128 | 128   | 3     | 1.48e-5 | 7.79e-3 |
| 32    | 3     | 0.25  | 128 | 128   | 4     | 1.34e-5 | 7.18e-3 |
| 32    | 3     | 0.25  | 256 | 256   | 3     | 1.56e-5 | 8.10e-3 |

The quantities  $W_1$ ,  $D_1$ ,  $P_1$ ,  $O$ ,  $W_3$  and  $D_3$  are defined in Section 6.1.4. We change only the hyperparameters of the dense layers as specified in columns  $W_3$  and  $D_3$ . All the other hyperparameters are the same as those reported in Table 8

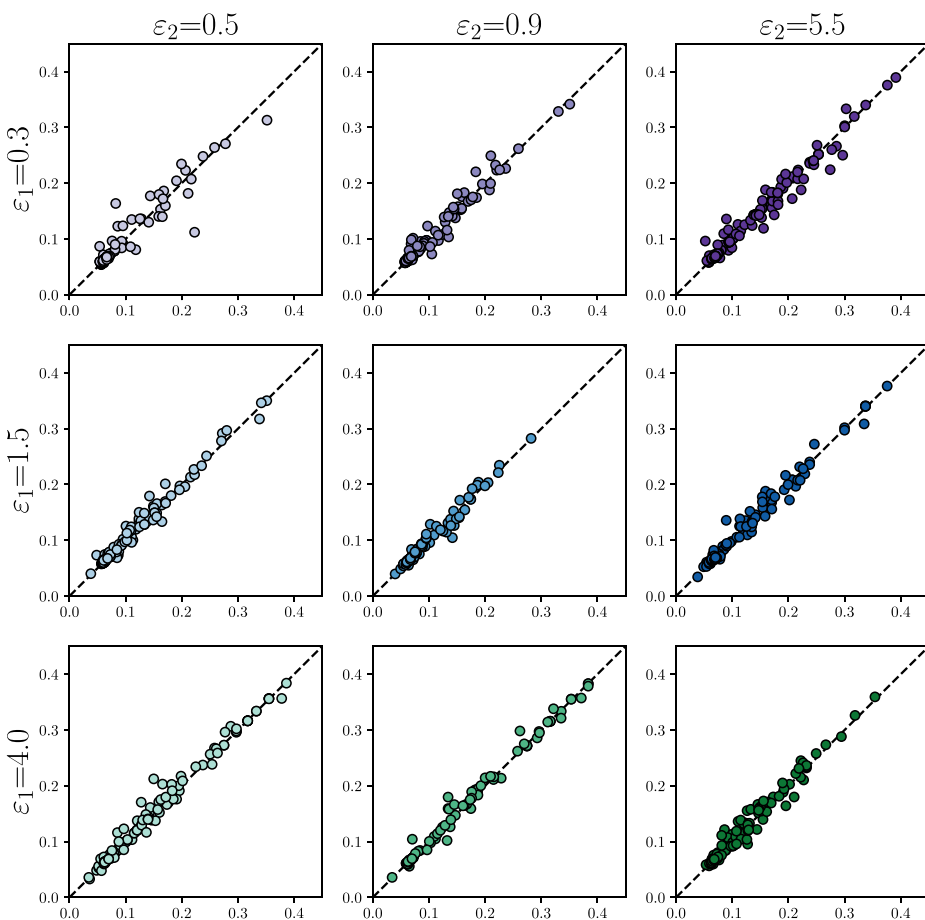
**Table 11** Computed loss (MSE) and MAE for different ANNs architectures (with one convolutional layer) trained with dataset 3

| $W_1$ | $D_1$ | $P_1$ | $O$ | $W_3$ | $D_3$ | loss    | MAE     |
|-------|-------|-------|-----|-------|-------|---------|---------|
| 16    | 4     | 0.25  | 128 | 128   | 4     | 1.32e-4 | 7.29e-3 |
| 16    | 4     | 0.50  | 128 | 128   | 4     | 1.56e-4 | 7.98e-3 |
| 16    | 5     | 0.25  | 128 | 128   | 4     | 1.54e-4 | 7.77e-3 |
| 16    | 3     | 0.50  | 128 | 128   | 4     | 1.51e-4 | 7.86e-3 |
| 24    | 2     | 0.25  | 128 | 128   | 4     | 1.53e-4 | 7.71e-3 |
| 24    | 2     | 0.50  | 128 | 128   | 4     | 1.52e-4 | 7.76e-3 |
| 24    | 3     | 0.25  | 128 | 128   | 4     | 1.40e-4 | 7.34e-3 |
| 24    | 4     | 0.50  | 128 | 128   | 4     | 1.60e-4 | 7.98e-3 |
| 32    | 3     | 0.25  | 128 | 128   | 4     | 1.34e-4 | 7.18e-3 |
| 32    | 3     | 0.50  | 128 | 128   | 4     | 1.47e-5 | 7.75e-3 |
| 40    | 2     | 0.25  | 128 | 128   | 4     | 1.27e-4 | 7.30e-3 |
| 40    | 3     | 0.25  | 128 | 128   | 4     | 1.32e-4 | 7.18e-3 |

The quantities  $W_1$ ,  $D_1$ ,  $P_1$ ,  $O$ ,  $W_3$  and  $D_3$  are defined in Section 6.1.4. Here, we change only the hyperparameters of the convolutional layer. Not specified hyperparameters are the same of Table 8

with 40 hidden units seems to be the best choice. The architecture that we choose for the model is the second to last of Table 11. Employing training with batch size 32, the Adam optimizer and early stopping (up to 1000 epochs), we obtain a loss on the test “dataset 3” of  $8.83 \cdot 10^{-5}$  and MAE  $4.84 \cdot 10^{-3}$ . On the test “dataset 1” we achieve a loss of  $7.73 \cdot 10^{-5}$  and a MAE of  $4.48 \cdot 10^{-3}$  and on test “dataset 2” we obtained a loss of  $1.39 \cdot 10^{-4}$  and a MAE of  $6.51 \cdot 10^{-3}$ . The predictions are reported in Fig. 11. Figure 9 (center) shows a histogram of the performance gain  $P$ . In particular, in 20% of the cases  $P \geq 37\%$ . Table 16 (second row) summarizes the results.

*Remark 1* We observe that the difference in predictions shown in Figs. 10 and 11 are significant. This begs the question of how to assess the quality of a training dataset. Unfortunately, it is very hard to assess the quality of the dataset a-priori. Indeed, this question is equivalent to predict the neural network generalization, which is still an open question in the field of



**Fig. 11** Prediction of the model trained with dataset 3 on the test dataset 2. Each plot show data obtained from a fixed combination of  $\varepsilon_1$  and  $\varepsilon_2$ . Namely,  $\varepsilon_1$  is constant for subplots on the same row and  $\varepsilon_2$  is constant for subplots on the same column of the plot grid. On the x-axis there is the true value of  $\rho$ , on the y-axis the predicted value

ML. However, in the case of very large datasets, it could be useful to train a small ANN on a small sample of the dataset and validate the results.

### 6.4 Test Case 3: Diffusion Coefficient with Different Values on Each Tile

In this case, we test the prediction capabilities of AMG-ANN whenever the diffusion coefficient shows a more complicated pattern. Namely, we generalize the definition of the diffusion coefficient  $\mu$  in the following way. Let  $\text{size} \in \mathbb{N}$  be a positive integer indicating the size of the pattern (e.g. in Fig. 1, from left to right,  $\text{size} = 2, 2, 4, 4$  since the patterns are two stripes,  $2 \times 2$  checkerboard, four stripes and  $4 \times 4$  checkerboard, respectively). Let  $\text{mode} = 1, 2$  be an integer that indicates if the pattern is at stripes or checkerboard like, respectively. These two parameters determine a partition  $\{\Omega_i\}_{i=1, \dots, \text{size}^{\text{mode}}}$  of the domain  $\Omega$ . On each element of the partition  $\Omega_i$  we set  $\mu(x, y) = 10^{(\epsilon)_i}$  (constant), where  $\epsilon \in \mathbb{R}^{\text{size}^{\text{mode}}}$  is a given vector of parameters. What just described is synthesized in Algorithm 5. Notice that if  $\text{size}$  is not a power of two, the pattern of  $\mu$  is not aligned with the mesh.

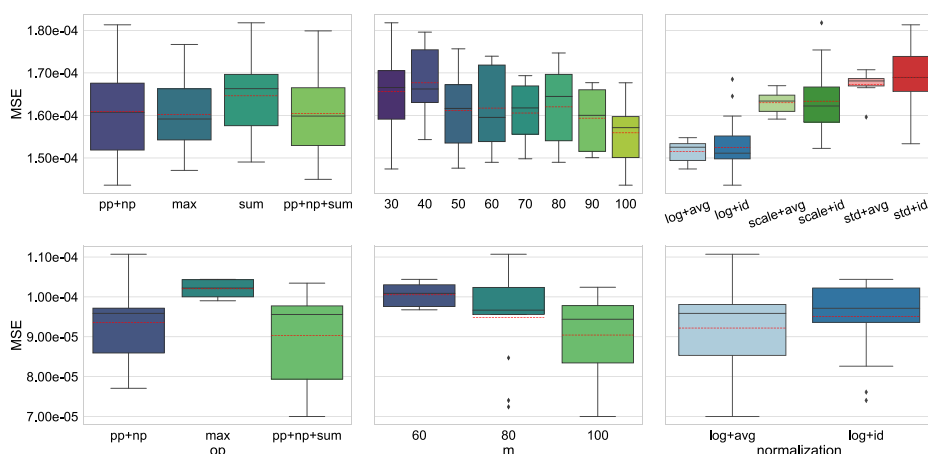
We choose  $f$  of (3) such that  $u(x, y) = \sin(\text{size}\pi x/2) \sin(\text{size}\pi y/2)$  if  $\text{size}$  is odd and  $u(x, y) = \cos(\text{size}\pi x/2) \cos(\text{size}\pi y/2)$  otherwise. Varying  $\text{size} = 2, \dots, 10$ ,  $\text{mode} = 1, 2$  and  $\epsilon$  by sampling its component at random with uniform distribution in the interval  $(-20, 20)$ , we create a dataset with about 200 000 samples ( $\theta$  and  $h$  vary in the same range as defined in Section 5).

#### 6.4.1 Tuning of the Hyperparameters of the Pooling Operation

To tune the hyperparameters  $\text{op}$ ,  $\text{normalize}$  and  $m$  we consider a subset of the dataset consisting of about 15 000 samples. By stacking together  $c$  views  $\hat{V}$  obtained with different  $\text{op}$  we can obtain a tensor  $\hat{V}' \in \mathbb{R}^{m \times m \times c}$  that can be interpreted as a multi-channel image. Namely, we consider four possible approaches summarized in Table 12. The first two rows represent the most used approaches in the computer vision field, whereas the argument for the approach in the third row comes from (1) where you can see the relevance of splitting the values into the positive and negative part. The fourth approach just combines the information of the first three together. We also consider all the six possible choices of  $\text{normalize}$  reported in Table 5 and eight values of  $m = 30, 40, \dots, 100$ . Thus, there are 192 combinations of hyperparameter  $m$ ,  $\text{op}$  and  $\text{normalize}$ . For each one of this choices we train a network with learning rate 0.001, batch size 32, 1 convolutional layer with no dropout and  $W_1 = 32$ ,  $D_1 = 2$ ,  $W_3 = 64$ ,  $D_3 = 3$ . Figure 12 shows the boxplot of the loss of the model

**Table 12** Summary of the possible ways to obtain  $\hat{V}' \in \mathbb{R}^{m \times m \times c}$  by stacking together  $\{\hat{V}_i = \text{normalize}(\text{pooling}(A_h, m, \text{op}_i))\}_{i=1, \dots, c}$ , where  $c$  and  $\{\text{op}_i\}_{i=1, \dots, c}$  are defined in the second and third columns, respectively

| Name      | #Channels $c$ | List of $\text{op}$                                                                                                                                      |
|-----------|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| sum       | 1             | $\text{op}(v_1, v_2) = v_1 + v_2$                                                                                                                        |
| max       | 1             | $\text{op}(v_1, v_2) = \max\{v_1,  v_2 \}$                                                                                                               |
| pp+np     | 2             | $\text{op}_1(v_1, v_2) = \max\{\max\{0, v_2\}, v_1\},$<br>$\text{op}_2(v_1, v_2) = \max\{\max\{0, -v_2\}, v_1\}$                                         |
| pp+np+sum | 3             | $\text{op}_1(v_1, v_2) = \max\{\max\{0, v_2\}, v_1\},$<br>$\text{op}_2(v_1, v_2) = \max\{\max\{0, -v_2\}, v_1\},$<br>$\text{op}_3(v_1, v_2) = v_1 + v_2$ |



**Fig. 12** Boxplot (with mean in red) representing the effect of different choices of the hyperparameters  $op$  (left),  $m$  (center) and  $normalize$  (right) on the test loss (MSE). The training is stopped after 100 epochs (top row), 200 epochs (bottom row)

trained for 100 epochs (top row) and 200 epochs (bottom row), grouped by hyperparameter category. From these results it seems that the choices  $normalize = \log+id$  and  $normalize = \log+avg$  provide the best results. Concerning the choice of  $op(\cdot, \cdot)$ , it seems that  $op = pp+np+sum$  leads to better results. The view size  $m$  seems to be inversely proportional to the loss. However, comparing  $m = 60$  and  $m = 100$  when the model is trained for 200 epochs, we notice that, on average, the loss is only about 10% larger even if the view is 64% smaller.

#### 6.4.2 Test Case 3.a: Predictions on 15 000 Samples

We train our model on the reduced dataset containing “only” 15 000 samples. After tuning the architecture of the neural network, we train three different models for 500 epochs. Table 13 shows the results. The choices  $normalize = \log+id$  and  $normalize = \log+avg$  produce similar results as before. We can conclude that  $m = 100$  seems better, but the improvement on the loss is only about 10% w.r.t.  $m = 80$ . Thus  $m = 80$  could be a reasonable choice to save time both in the offline and online phases. The second model results into the lowest loss and MAE, its performance is reported in Table 16. Moreover, we observe that in 20% of the cases it has a performance gain  $P \geq 27\%$ .

**Table 13** Case 3.a: computed loss (MSE) and MAE for different choices of the hyperparameters  $op$ ,  $m$  and  $normalize$  hyperparameters

| $op$      | $m$ | $normalize$ | $W_1$ | $D_1$ | $P_1$ | $O$ | $W_3$ | $D_3$ | loss    | MAE     |
|-----------|-----|-------------|-------|-------|-------|-----|-------|-------|---------|---------|
| pp+np+sum | 100 | log+avg     | 40    | 3     | 0     | 128 | 128   | 5     | 4.31e-5 | 3.65e-3 |
| pp+np+sum | 100 | log+id      | 40    | 3     | 0     | 128 | 128   | 5     | 3.87e-5 | 3.47e-3 |
| pp+np+sum | 80  | log+avg     | 40    | 3     | 0     | 128 | 128   | 5     | 4.07e-5 | 3.57e-3 |

All the other hyperparameters are the same as those reported in Table 8

**Table 14** Comparison of the performance of two fixed models when using MSE or MAE as a loss

|         | Loss | MAE     | MSE     | $PB$  | $P$ (avg/median) |       | $P_{<0}$ (avg/median) |        |
|---------|------|---------|---------|-------|------------------|-------|-----------------------|--------|
| Model 1 | MSE  | 1.00e-3 | 2.95e-4 | 62.3% | 3.14%            | 2.67% | -13.1%                | -7.91% |
| Model 2 | MSE  | 9.67e-4 | 2.71e-4 | 62.5% | 3.55%            | 2.19% | -12.3%                | -7.74% |
| Model 1 | MAE  | 9.35e-4 | 2.83e-4 | 63.7% | 3.69%            | 3.56% | -13.4%                | -7.47% |
| Model 2 | MAE  | 1.00e-3 | 3.18e-4 | 61.8% | 2.85%            | 2.86% | -14.1%                | -8.69% |

The two models are the second and third of Table 15. They were trained for 20 epochs, batch size 32, default Tensorflow learning rate of Adam. The quantities  $PB$ ,  $P$  and  $P_{<0}$  are defined in Section 6.1.4

### 6.4.3 Test Case 3.b: Predictions on 200 000 Samples

In this section we train our model with the full dataset containing 200 000 samples. We use the following choices as hyperparameters  $m = 50$ , `normalize = scale+avg` and `op = sum` since they provide a good compromise between accuracy and efficiency for tuning the architecture of the ANN.

First, we check if using as loss function MSE or MAE makes any difference on the final performance: we fix two models architectures and train the ANNs changing only the loss function. Results are reported in Table 14: there seems not to be any significant difference between the two.

We then perform an hyperparameter optimization on the model architecture: we train different models for 40 epochs and check their performance in terms of loss. We test different dropout probabilities, different sizes and depths for the convolution filters and different architectures for the dense part. We report some results in Table 15, where we can see that larger models, w.r.t. the ones in Table 11, perform better. This is expected since we have a rather larger dataset.

We choose as architecture of our model the one reported in the seventh row of Table 15. Using as hyperparameters of the view `op = pp+np+sum`,  $m = 100$ , `normalize = log+avg`, the model is trained for 500 epochs (validation loss was still decreasing when training ended). It reaches a loss of  $6.95 \cdot 10^{-5}$  MSE ( $4.69 \cdot 10^{-3}$  MAE). We show its performance in Table 16 (fourth row). The histogram of the performance is reported in Fig. 9 (right). Even if its performance is not as good as the one of the model obtained in Section 6.3, we would like to point out that there is still a margin of improvement for the hyperparameters optimization step and more epochs could be used.

---

```

1  $j \leftarrow 1$ ;
2 for  $i \leftarrow 1$  to  $mode$  do
3    $j \leftarrow j + \lfloor ((\mathbf{x})_i + 1) size/2 \rfloor size^{i-1}$ ;
4 end
5  $\mu \leftarrow 10^{(\epsilon)j}$ ;
```

---

**Algorithm 5** Diffusion coefficient  $\mu$  in a given point  $\mathbf{x} \in \Omega = (-1, 1)^2$ , fixed the parameters  $mode$ ,  $size$  and  $\epsilon \in \mathbb{R}^{size^{mode}}$ .  $\mu = \mu(\mathbf{x}; mode, size, \epsilon)$ .

**Table 15** Computed loss (MSE) and MAE on for different architectures of the ANN trained on the dataset of Section 6.4 for 40 epochs

| $W_1$ | $D_1$ | $P_1$ | $O$ | $W_3$ | $D_3$ | Loss    | MAE     |
|-------|-------|-------|-----|-------|-------|---------|---------|
| 32    | 2     | 0.25  | 128 | 128   | 4     | 3.64e-3 | 1.24e-2 |
| 32    | 3     | 0.25  | 128 | 128   | 4     | 2.22e-3 | 9.35e-3 |
| 32    | 4     | 0.25  | 128 | 128   | 4     | 2.16e-3 | 9.16e-3 |
| 32    | 5     | 0.25  | 128 | 128   | 4     | 2.26e-3 | 9.48e-3 |
| 40    | 2     | 0.25  | 128 | 128   | 4     | 2.45e-3 | 9.90e-3 |
| 40    | 3     | 0.25  | 128 | 128   | 4     | 2.12e-3 | 9.22e-3 |
| 40    | 3     | 0.25  | 128 | 128   | 5     | 1.98e-3 | 8.75e-3 |
| 40    | 4     | 0.25  | 128 | 128   | 4     | 2.22e-3 | 9.31e-3 |
| 64    | 3     | 0.25  | 128 | 128   | 4     | 2.19e-3 | 9.31e-3 |
| 64    | 4     | 0.25  | 128 | 128   | 4     | 3.16e-3 | 1.01e-2 |

The quantities  $W_1$ ,  $D_1$ ,  $P_1$ ,  $O$ ,  $W_3$  and  $D_3$  are defined in Section 6.1.4. Not specified hyperparameters are the same of Table 14

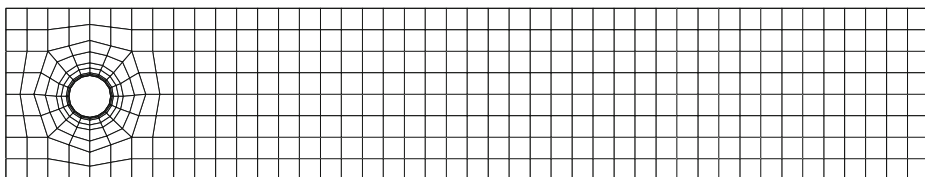
## 6.5 Test Case 4: the Stationary Stokes Problem

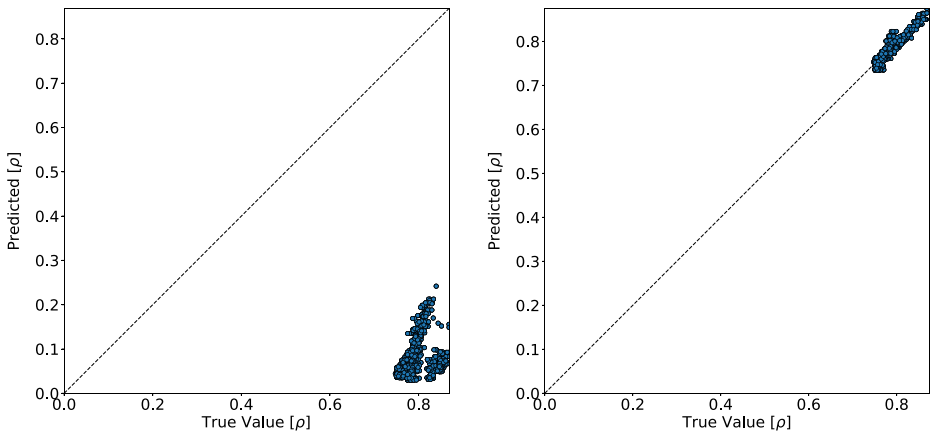
Finally, we show how we can extend the prediction capabilities of the ANN trained in the previous section in the case we consider a different model problem. Namely, we consider the model problem defined in (7). Namely, we consider the channel flow around a cylinder with parabolic inflow profile in  $\Omega = (0, 2.2) \times (0, 0.41) \setminus B_r(0.2, 0.2) \subset \mathbb{R}^2$  with  $r = 0.05$  where  $B_r(x, y)$  is the ball of radius  $r$  centered in  $(x, y) \in \mathbb{R}^2$ . We define  $\Gamma_0 = [0, 2.2] \times \{0, 0.41\} \cup \partial B_r(0.2, 0.2)$ ,  $\Gamma_{in} = 0 \times [0, 0.41]$ ,  $\Gamma_{out} = 2.2 \times [0, 0.41]$ . We report in Fig. 13 a graphical representation of the computational domain together with a computational mesh. We solve its discrete formulation (8) with MINRES preconditioned with

$$P^{-1} = \begin{pmatrix} A_h & 0 \\ 0 & M_h \end{pmatrix}^{-1} = \begin{pmatrix} A_h^{-1} & 0 \\ 0 & M_h^{-1} \end{pmatrix},$$

where  $M_h$  is the mass matrix in the pressure space. For the approximation of the velocity block  $A_h$  we will perform a single AMG V-cycle: our AMG-ANN will be applied to this block.

Then, we build a dataset with 3600 samples by varying  $\nu = 0.001, 0.1, 10$ ,  $U = 0.00001, 0.001, 0.1, 10$ ,  $h = 0.14/2^i$  with  $i = 0, 1, 2, 3, 4, 5$ . Our baseline is the model trained in the Section 6.4. Using transfer learning, we would like to extend its applicability range to the

**Fig. 13** Representation of the domain of the Stokes problem (7) with a quadrilateral mesh  $\mathcal{T}_h$



**Fig. 14** *Left.* Predictions on the Stokes problem dataset defined in Section 6.5 of the model trained in Section 6.4 (i.e. with only Laplace problems). *Right.* The predictions of the same model after training for an extra 20 epochs on a dataset composed at 50% by Stokes problems and 50% by samples of the dataset defined in Section 6.4. Not specified hyperparameters are the same of Table 14

Stokes problem. To this end, using the weights (and architecture) of the pre-trained model obtained in Section 6.4, we train it with a dataset composed at 50% by Stokes problem samples and 50% by data from the dataset we build in Section 6.4 (i.e. the dataset used to pre-train the model). Indeed, we keep some of the old data to avoid catastrophic forgetting [37], i.e. the tendency of ANNs to forget how to perform a task upon learning new information. We train the model only for 20 epochs, we employ early stopping and a 80%-20% training-validation split. In Fig. 14 we report the prediction of the model before and after this step of training. At the end of the training the model has a loss of  $1.52 \cdot 10^{-4}$  MSE (MAE  $9.09 \cdot 10^{-3}$ ). We report its performance in Table 16. We remark that, in this case, a performance gain  $P$  of 3% (in terms of  $\rho$ ) results into about 30% performance gain in terms of elapsed CPU time. Meanwhile on the full original dataset we still maintain a  $PB > 50\%$  and  $P > 0$  both in mean and median. Thus, with a small computational effort, we succeeded in extending the knowledge of our model.

**Table 16** Evaluation of the performance of the best model for each case

|          | $PB$   | $P$ (avg/median) |        | $P/P_{\text{MAX}}$ (avg/median) |        | $P_{<0}$ (avg/median) |        |
|----------|--------|------------------|--------|---------------------------------|--------|-----------------------|--------|
| Case 1   | 96.61% | 17.27%           | 4.11%  | 86.96%                          | 99.17% | -1.65%                | -0.63% |
| Case 2   | 89.13% | 14.92%           | 2.81%  | 80.09%                          | 96.01% | -3.72%                | -2.04% |
| Case 3.a | 88.60% | 15.03%           | 13.60% | 72.06%                          | 82.24% | -5.47%                | -2.25% |
| Case 3.b | 80.65% | 12.60%           | 10.94% | 65.82%                          | 69.79% | -6.00%                | -3.91% |
| Case 4   | 90.16% | 2.45%            | 1.24%  | 79.62%                          | 90.76% | -0.36%                | -0.39% |

The models and cases are defined in Section 6.1.4, Section 6.3 and Section 6.4, Section 6.5 respectively. The quantities  $PB$ ,  $P$ ,  $P_{\text{MAX}}$  and  $P_{<0}$  are defined at the end of Section 6.1.4

## 7 Conclusions

In this work, we developed an ANN-based approach to enhance the computational efficiency of the AMG methods, i.e. to accelerate their performance. In particular, we accurately predicted the value of the strong threshold parameter  $\theta$  that maximizes the performance with respect to the matrix  $A_h$  of the linear system to be solved. In order to be able to apply the model independently of the matrix of the linear system, we introduced a pooling operator. We measured the efficiency of the AMG method using the approximate convergence factor and we designed a model that predicts its value. In this way, we are able to choose the strong threshold parameter that minimizes the predicted approximated convergence factor. Moreover, we have shown that, as expected, the approximated convergence factor of the AMG method is strictly correlated to the elapsed CPU time during the application of the AMG method to the linear system solution, thus demonstrating that it provides a good measure of the performance of the solver. This a priori, optimal selection of the strong threshold parameter allows us to efficiently choose a value of  $\theta$  that significantly decreases the elapsed CPU time with respect to the “classical” value. We introduced a set of indicators to measure the performance of our model: we show that if the dataset is smaller than 15 000 samples our model is better than using the literature value of  $\theta$  in about 90% of the cases with on average a 15% gain in performance. On the other hand, from our computations, it seems that in case of large datasets more work on the tuning of the hyperparameter is needed. Finally, we have addressed a generalization test case moving from the elliptic scalar differential problem to the Stokes system exploiting the so called transfer learning. The preliminary results are encouraging and further investigation will be the subject of further research.

Possible further developments also include: using the ANN to optimize the value of other AMG parameters, such as the maximum row sum parameter, the choice of whether using W-cycles or V-cycles or the number of levels of aggressive coarsening. Possible other improvements include tuning of the hyperparameters of the model; testing a wider range of differential models, in particular in three-dimensional configuration.

**Acknowledgements** P.F.A and L.D. are members of the INdAM Research group GNCS. P.F.A has been partially funded by the research projects PRIN17 (n. 201744KLJL) and PRIN 2020 (n. 20204LN5N5), funded by Italian Ministry of University and Research (MUR). L.D. has been partially funded by the research project PRIN 2020 (n. 20204LN5N5) funded by MUR.

## References

1. Abhyankar, S., Brown, J., Constantinescu, E.M., Ghosh, D., Smith, B.F., Zhang, H.: PETSc/TS: a modern scalable ODE/DAE solver library. arXiv:1806.01437 (2018)
2. Antonietti, P.F., Dassi, F., Manzuzzi, E.: Machine learning based refinement strategies for polyhedral grids with applications to Virtual Element and polyhedral discontinuous Galerkin methods. *J. Comput. Phys.* In press (2022)
3. Antonietti, P.F., Manzuzzi, E.: Refinement of polygonal grids using convolutional neural networks with applications to polygonal discontinuous Galerkin and virtual element methods. *J. Comput. Phys.* **452**, 110900 (2022)
4. Antonietti, P.F., Melas, L.: Algebraic multigrid schemes for high-order nodal discontinuous Galerkin methods. *SIAM J. Sci. Comput.* **42**, A1147–A1173 (2020)
5. Arndt, D., Bangerth, W., Clevenger, T.C., Davydov, D., Fehling, M., Garcia-Sanchez, D., Harper, G., Heister, T., Heltai, L., Kronbichler, M., Kynch, R.M., Maier, M., Pelteret, J.-P., Turcksin, B., Wells, D.: The deal.II library, version 9.1. *J. Numer. Math.* **27**, 203–213 (2019)

6. Ashby, S.F., Falgout, R.D.: A parallel multigrid preconditioned conjugate gradient algorithm for groundwater flow simulations. *Nuclear Sci. Eng.* **124**, 145–159 (1996)
7. Baker, A.H., Falgout, R.D., Kolev, T.V., Yang, U.M.: Scaling Hypr's multigrid solvers to 100,000 cores. In: Berry, M.W., et al. (eds.) *High-Performance Scientific Computing*, pp. 261–279. Springer, London (2012)
8. Baker, A.H., Kolev, Tz.V., Yang, U.M.: Improving algebraic multigrid interpolation operators for linear elasticity problems. *Numer. Linear Algebra Appl.* **17**, 495–517 (2010)
9. Bank, R., Falgout, R., Jones, T., Manteuffel, T.A., McCormick, S.F., Ruge, J.W.: Algebraic multigrid domain and range decomposition (AMG-DD/AMG-RD). *SIAM J. Sci. Comput.* **37**, S113–S136 (2015)
10. Bastian, P., Blatt, M., Scheichl, R.: Algebraic multigrid for discontinuous Galerkin discretizations of heterogeneous elliptic problems. *Numer. Linear Algebra Appl.* **19**, 367–388 (2012)
11. Boffi, D., Brezzi, F., Fortin, M.: *Mixed Finite Element Methods and Applications*. Springer Series in Computational Mathematics, Vol. 44. Springer, Berlin (2013)
12. Brandt, A., Livne, O.E.: *Multigrid Techniques*. SIAM, Philadelphia (2011)
13. Brannick, J., Brezina, M., MacLachlan, S., Manteuffel, T., McCormick, S., Ruge, J.: An energy-based AMG coarsening strategy. *Numer. Linear Algebra Appl.* **13**, 133–148 (2006)
14. Brezina, M., Cleary, A.J., Falgout, R.D., Henson, V.E., Jones, J.E., Manteuffel, T.A., McCormick, S.F., Ruge, J.W.: Algebraic multigrid based on element interpolation (AMGe). *SIAM J. Sci. Comput.* **22**, 1570–1592 (2001)
15. Brezis, H.: *Functional Analysis, Sobolev Spaces and Partial Differential Equations*. Springer, New York (2010)
16. Bui, Q.M., Wang, L., Osei-Kuffuor, D.: Algebraic multigrid preconditioners for two-phase flow in porous media with phase transitions. *Adv. Water Resour.* **114**, 19–28 (2018)
17. Cleary, A.J., Falgout, R.D., Henson, V.E., Jones, J.E., Manteuffel, T.A., McCormick, S.F., Miranda, G.N., Ruge, J.W.: Robustness and scalability of algebraic multigrid. *SIAM J. Sci. Comput.* **21**, 1886–1908 (2000)
18. Discacciati, N., Hesthaven, J.S., Ray, D.: Controlling oscillations in high-order discontinuous Galerkin schemes using artificial viscosity tuned by neural networks. *J. Comput. Phys.* **409**, 109304 (2020)
19. Falgout, R., Vassilevski, S.: On generalizing the algebraic multigrid framework. *SIAM J. Sci. Comput.* **42**, 1669–1693 (2004)
20. Falgout, R., Yang, U.: Hypr: a library of high performance preconditioners. In: Sloot, P.M.A., et al. (eds.) *Computational science—ICCS 2002*. Lecture Notes in Computer Science, vol. 2331, pp. 632–641. Springer, Berlin (2002)
21. Fresca, S., Dede', L., Manzoni, A.: A comprehensive deep learning-based approach to reduced order modeling of nonlinear time-dependent parametrized PDEs. *J. Sci. Comput.* **87**, 61 (2021)
22. Goodfellow, I., Bengio, Y., Courville, A.: *Deep Learning*. The MIT Press, Massachusetts (2016)
23. Gottschalk, H., Kahl, K.: Coarsening in algebraic multigrid using gaussian processes. *Electron. Trans. Numer. Anal.* **54**, 514–533 (2021)
24. Greenfeld, D., Galun, M., Basri, R., Yavneh, I., Kimmel, R.: Learning to optimize multigrid PDE solvers. In: *International Conference on Machine Learning*, pp. 2415–2423. PMLR (2019)
25. He, K., Zhang, X., Ren, S., Sun, J.: Delving deep into rectifiers: surpassing human-level performance on imagenet classification. In: *Proceedings of the IEEE International Conference on Computer Vision*, pp. 1026–1034 (2015)
26. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778 (2016)
27. Heinlein, A., Klawonn, A., Lanser, M., Weber, J.: Combining machine learning and adaptive coarse spaces—a hybrid approach for robust FETI-DP methods in three dimensions. *SIAM J. Sci. Comput.* **43**, S816–S838 (2021)
28. Henson, V.E., Vassilevski, P.S.: Algebraic multigrid preconditioners for two-phase flow in porous media with phase transitions. *SIAM J. Sci. Comput.* **23**, 629–650 (2001)
29. Hesthaven, J.S., Ubbiali, S.: Non-intrusive reduced order modeling of nonlinear problems using neural networks. *J. Comput. Phys.* **363**, 55–78 (2018)
30. Hu, J., Shen, L., Sun, G.: Squeeze-and-excitation networks. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 7132–7141 (2018)
31. Hughes, T.J.R.: *The Finite Element Method: Linear Static and Dynamic Finite Element Analysis*. Courier Corporation (2012)

32. Ioffe, S., Szegedy, C.: Batch normalization: accelerating deep network training by reducing internal covariate shift. arXiv:[1502.03167](https://arxiv.org/abs/1502.03167) (2015)
33. Janssens, M., Hulshoff, S.: Advancing artificial neural network parameterization for atmospheric turbulence using a variational multiscale model. *J. Adv. Model. Earth Syst.* **14**, e2021MS002490 (2021)
34. Jones, J., Lee, B.: A multigrid method for variable coefficient Maxwell's equations. *SIAM J. Sci. Comput.* **27**, 1689–1708 (2006)
35. Katrutsa, A., Daulbaev, T., Oseledets, I.: Black-box learning of multigrid parameters. *J. Comput. Appl. Math.* **368**, 112524 (2020)
36. Kingma, D.P., Ba Adam, J.: A method for stochastic optimization. arXiv:[1412.6980](https://arxiv.org/abs/1412.6980) (2014)
37. Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A.A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., Hassabis, D., Clopath, C., Kumaran, D., Hadsell, R.: Overcoming catastrophic forgetting in neural networks. *Proc. National Acad. Sci.* **114**, 3521–3526 (2017)
38. Krizhevsky, A., Sutskever, I., Hinton, G.E.: Imagenet classification with deep convolutional neural networks. In: *Advances in Neural Information Processing Systems*, pp. 1097–1105 (2012)
39. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. *Proc. IEEE* **86**, 2278–2324 (1998)
40. Li, R., Zhang, C.: Efficient parallel implementations of sparse triangular solves for GPU architectures. In: *Proceedings of the 2020 SIAM Conference on Parallel Processing for Scientific Computing*, pp. 106–117. SIAM (2020)
41. Lions, J.L., Magenes, E.: *Non-Homogeneous Boundary Value Problems and Applications*. Springer, Berlin (1972)
42. Mishra, S.: A machine learning framework for data driven acceleration of computations of differential equations. *Math. Eng.* **1**, 118–146 (2019)
43. Neittaanmäki, P., Repin, S.: Artificial intelligence and computational science. In: Tuovinen, T., Periaux, J., Neittaanmäki, P. (eds.) *Computational Sciences and Artificial Intelligence in Industry*. Intelligent Systems, Control and Automation: Science and Engineering, vol. 76, pp. 27–35. Springer, Cham (2022)
44. Quarteroni, A.: *Numerical Models for Differential Problems*. MS&A, vol. 16. Springer, Cham (2017)
45. Quarteroni, A., Valli, A.: *Numerical Approximation of Partial Differential Equations*. Springer, Berlin (1994)
46. Raissi, M., Perdikaris, P., Karniadakis, G.E.: Machine learning of linear differential equations using Gaussian processes. *J. Comput. Phys.* **348**, 683–693 (2017)
47. Raissi, M., Perdikaris, P., Karniadakis, G.E.: Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.* **378**, 686–707 (2019)
48. Regazzoni, F., Dede', L., Quarteroni, A.: Machine learning for fast and reliable solution of time-dependent differential equations. *J. Comput. Phys.* **397**, 108852 (2019)
49. Ruge, J.W., Stüben, K.: 4. Algebraic multigrid. In: McCormick, S.F. (ed.) *Multigrid Methods*, pp. 73–130. SIAM (1987)
50. Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A.C., Fei-Fei, L.: Imagenet large scale visual recognition challenge. *Int. J. Comput. Vis.* **115**, 211–252 (2015)
51. Seabold, S., Perktold, J.: Statsmodels: Econometric and Statistical Modeling with Python. In: *9th Python in science conference* (2010)
52. Siefert, C., Tuminaro, R., Gerstenberger, A., Scovazzi, G., Collis, S.S.: Algebraic multigrid techniques for discontinuous Galerkin methods with varying polynomial order. *Comput. Geosci.* **18**, 597–612 (2014)
53. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., Salakhutdinov, R.: Dropout: a simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.* **15**, 1929–1958 (2014)
54. Stüben, K.: An introduction to algebraic multigrid. In: *Multigrid*, Chapter A pp. 413–532. Elsevier Science (2001)
55. Stüben, K.: A review of algebraic multigrid. *J. Comput. Appl. Math.* **128**, 281–309 (2001)
56. Sutskever, I., Martens, J., Dahl, G., Hinton, G.: On the importance of initialization and momentum in deep learning. In: *Proceedings of the 30th International Conference on Machine Learning*, pp. 1139–1147 (2013)
57. Tassi, T., Zingaro, A., Dede', L.: A machine learning approach to enhance the SUPG stabilization method for advection-dominated differential problems. *Math. Eng.* **5**(2), 1–26 (2023)

58. Tieleman, T., Hinton, G.: Lecture 6.5-rmsprop: divide the gradient by a running average of its recent magnitude. COURSERA: Neural Netw. Mach. Learn. **4**, 26–31 (2012)
59. Waskom, M.L.: Seaborn: statistical data visualization. J. Open Source Softw. **6**, 3021 (2021)
60. Webster, R.: An algebraic multigrid solver for navier-stokes problems. Int. J. Numer. Methods Fluids **18**, 761–780 (1994)
61. Xu, J., Zikatanov, L.: Algebraic multigrid methods. Acta Numer. **26**, 591–721 (2017)

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.